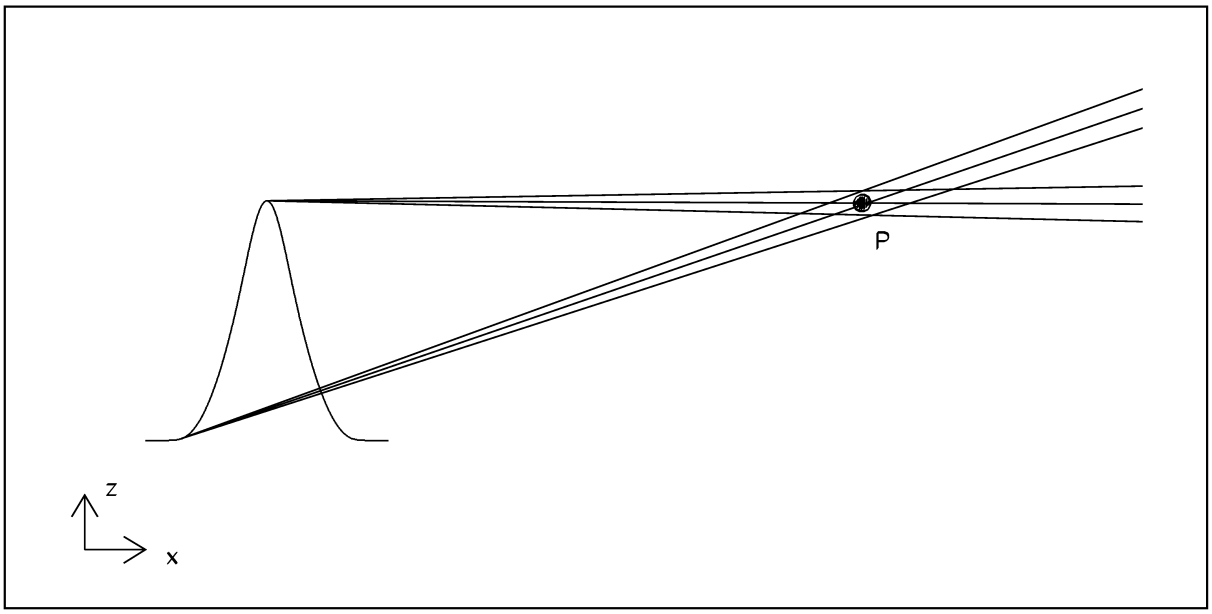
WAVE by Examples

Michael Scheer
Helmholtz-Zentrum Berlin

November 12, 2025



Contents

I Introduction

WAVE has been developed at BESSY over the years to track electrons through magnetic fields and to calculate spontanous synchrotron radiation [1]. This document illustrates the usage of WAVE and makes you familiar with it by following the given examples.

WAVE is mainly controlled by an ASCII input file named ***wave.in***. For each example, there is a script ***wave_example_n.sh*** to create ***wave.in*** from ***$WAVE/input/wave.examples*** , where ***n*** is the number of the example. Once ***wave.in*** is created by the corresponding script, you may compare ***$WAVE/input/wave.examples*** and ***wave.in*** to see, which variables have changed.

There is a graphical user interface (GUI), a Python script called ***WAVESHOP*** to setup ***wave.in***, to run WAVE , and to visualize the results. This is explained in the first example. However, not all options for WAVE are available in the GUI.

Once WAVE is installed, you should have a WAVE directory and shell variable WAVE pointing to it. It is assumed, that the user knows to edit a file and how to open a terminal, e.g. ***bash*** under Linux or powershell and under Windows. As an alternative one may run the Python shell ***idle***.

WAVE and the related software is published under the GNU General Public License. To get WAVE, please contact the author or get it from github:

git clone https://github.com/MicScheer/wave.git wave

II Getting Started

The examples in this document refer to a ***LINUX*** machine, however, they should be easy to understand and adapted to a ***WINDOWS*** computer.

The main differences are, that the slashes in the file and path names must be replaced by backslashes, shell scripts ($*.sh$) must be replaced by $*.bat$ files, and the system variables $\$V$ must be replaced by $\%V\%$.

II.1 Running WAVE

Under ***WINDOWS*** open e.g. a DOS command shell (cmd.exe), goto to the directory where WAVE is installed and excute ***bat\set_wave_environment.bat***. Then enter ***wave*** or ***waveshop***. But as mentioned before, we now refer to ***LINUX***.

First define a shell variable *$WAVE* containing the path to your WAVE -directory:

export WAVE=YOUR-WAVE-DIRECTORY

export WAVE_INCL=$WAVE

It is recommended to put these lines in a script, see e.g.

. $WAVE/shell/set_wave_environment.sh

and to excute it at the beginning of a WAVE -session.

To run WAVE the traditional way is to is to enter in a terminal

cd $WAVE/bin/stage

../bin/wave.exe

It is recommended to use the script

$WAVE/stage/wave

to run WAVE .

WAVE should run and write some output to your terminal (Fig. 1).

The program writes some output to the screen and the output text files ***wave.out*** and ***WAVE.mhb***. The file ***wave.out*** contains the most important in- and output of the run, while ***WAVE.mhb*** contains the histograms and n-tuples to be plotted. Please open these files and have a look...

If the program crashes, it must probably be recompiled in your environment by

python3 $WAVE/python/make_wave.py

if you have ***gfortran*** installed, or by

python3 $WAVE/python/make_wave_intel.py

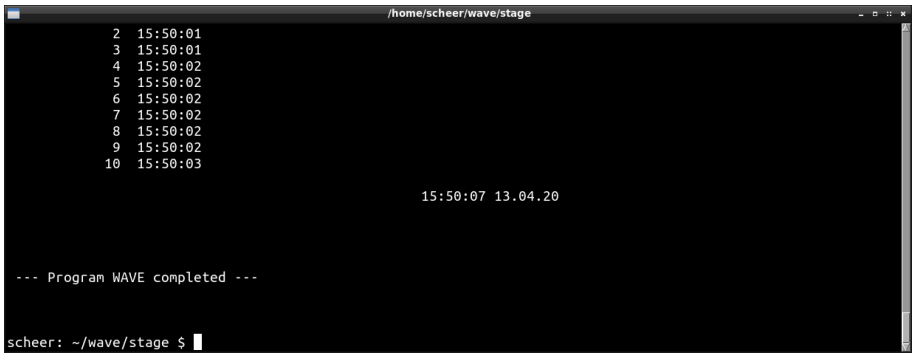


Fig. 1: Terminal output of *WAVE*

if you have the Intel compiler ***ifx*** installed.

If these Python scripts are run with an argument, they run in a verbose mode.

II.2 Plotting with Python

A convenient way to visualize the histograms and n-tuples is to use a Python scripts

python3 -i ../python/waveplot.py

First, we check the installation of Python. Enter:

python3

If you get a ***Python*** command prompt, ***Python3*** is installed. Note: Under ***WINDOWS*** the command is probably ***python*** instead of ***python3*** . However, the version $3.x$ of *Python* is required. You can leave the ***Python*** shell with:

quit()

If the test was successful, try the same with ipython3:

ipython3

If there is no command ***ipython3*** , you can install it (given pip3 is installed):

pip3 install ipython3

II.3 The Graphical User Interface *WAVESHOP*

There is a graphical user interfaces (GUI) based on ***Python*** and ***Tkinter*** .

To run the GUI *WAVESHOP* (Fig. 2), you need ***python3*** or ***ipython3***. In addition, the file ***waves.wvs*** must be present in the working directory. The ***Python*** script ***../python/waveshop.py*** is started by the command alias ***waveshop*** . Thus enter

WAVESHOP

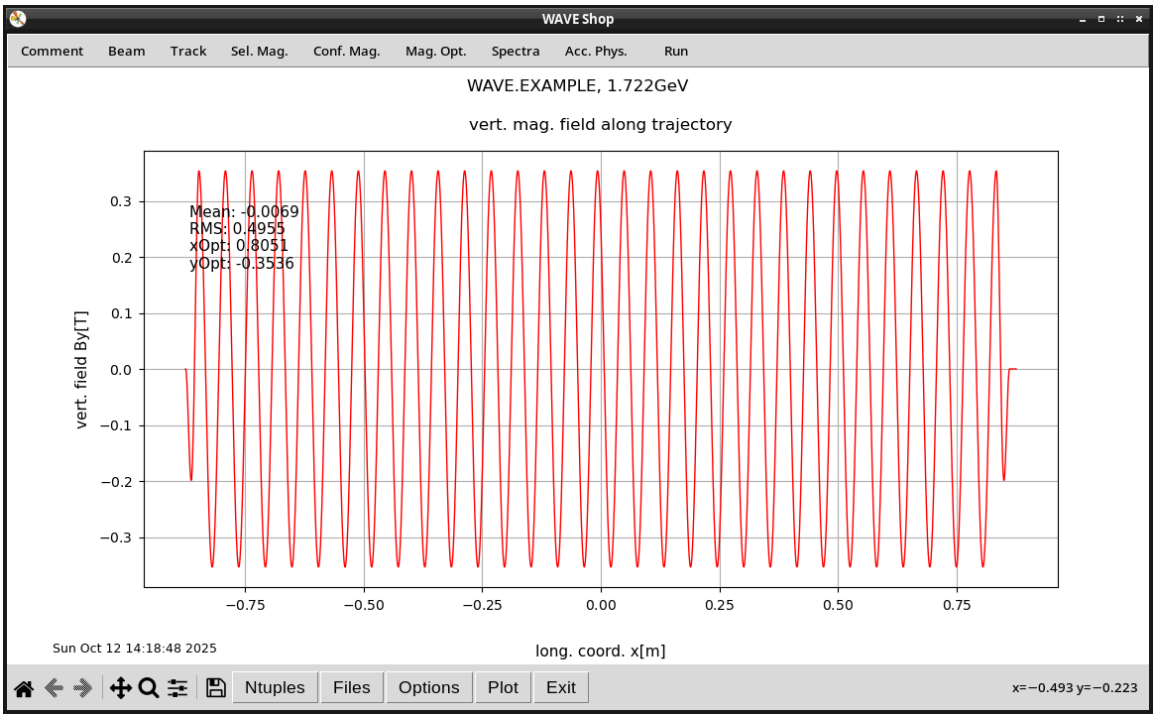


Fig. 2: Main window of *WAVESHOP* . The upper menu items are used to set-up ***wave.in*** and to run WAVE , the lower buttons to plot the results.

or, if the alias is not defined accordingly, by

python3 -i ../python/waveshop.py

II.4 The Input File *wave.in*

The GUI for WAVE covers many standard calculations, however, to get the most out of WAVE , it is necessary to study and understand the input file ***wave.in***.

We open ***wave.in*** with an editor and search ***$CONTRL*** . This keyword belongs to a ***FORTRAN*** namelist. A namelist begins with such a keyword and ends with ***$END*** . The file ***wave.in*** is actually nothing than a list of namelists.

A namelist is a list of variables, read by the program. The position of the variable in the namelist does'nt matter. Missing variables are initiliazed as zero or an empty string. For the name of the variables the ***FORTRAN*** conventions aplly, i.e. variables beginning with ***I, J, K, L, M, N*** are integer, all others are strings, floating point, or complexe numbers. The integer must not have a decimal point, while the floating point and complexe numbers onces must have. The same variable may occur several times within a namelist. Then the last value is accepted. If a whole namelist occurs several times, only the first one is read. Comments must begin with "!".

The namelist ***$CONTRL*** is the most important one of ***wave.in***. It controlls the program flow, the tracking, the field selection and so on.

```
F10 key ==> File   Edit   Search   Buffers   Windows   System   Help
!*******************************************************************************
 !
      !                              VERSION 3.03/00
 !
 $CONTRL
  !-------------------------- USER COMMENT ---------------------------

      CODE='WAVE.EXAMPLE VERSION 3.03/00'

  !--------------------------- MAIN MODES ----------------------------

        ! The undulator and wiggler modes should work for standard
        ! insertion devices. Reasonable settings for some parameters
        ! are taken (mainly in namelist COLLIN).
        ! Experienced users might prefere there own settings.

        IUNDULATOR=1    ! UNDULATOR MODE:

                        ! IBUNCH.EQ.0:
                        ! whole trajectory is taken as source of
                        ! synchroton light (ignoring input of
-------+  Jed: wave.in (Text) 58/3170  1   11:40am  ----------------------------
```

Fig. 3: First part of the input file *wave.in*

A short example: Look for a line, containing

KELLIP=1

This variables activates an analytical model of an elliptical undulator. The comment in this line mentions the namelist ***$ELLIPN***. Hence, we search ***$ELLIPN***.

```
XJed
F10 key ==> File   Edit   Search   Buffers   Windows   System   Help

$ELLIPN          ! helical undulator

      PARKELL=0.  ! if not zero, it is effective K-parameter of undulator
                  ! mag. fields B0ELLIPV and B0ELLIPH are adjusted
                  ! signs and ratio of B0ELLIPV and B0ELLIP are kept

      B0ELLIPV=0.353553390593   ! vertical peak field   [T]
      B0ELLIPH=0.353553390593   ! horizontal peak field   [T]

      XLELLIP=0.056   ! period length [m]
      PERELLIP=31.    ! number of periods
      ELLSHFT=0.25    ! shift of horiz. and vert. fields [periods]

      NHARMELL=0 ! if not zero, magnetic field is calculated such, that
                 ! the energy of the NHARMth harmonic is HARMELL
                 ! signs and ratio of B0ELLIPV and B0ELLIP are kept
                 ! OVERWRITES PARKELL!

      HARMELL=1000.  ! photon energy of considered harmonic [eV]
                     !-9999.: HARMELL=(FREQLOW+FREQHIG)/2 or
                     ! or = FREQLOW for single photon energy
                     !other negative values: HARMELL denotes wavelength [nm]

      XCENELL=0.0 ! longitudinal position of undulator center [m]
      ELLTAP=0.0 ! scaling factor for taper

      $END

-**----+  Jed: wave.in (Text) 1353/3325  1   4:09pm  ----------------------------
```

Fig. 4: The namelist ***ELLIPN*** of the elliptcal undulator model

Here you could change the parameters of the selected elliptical undulator.

Note: The GUI *WAVESHOP* can't handle multiple occurances of variables, i.e. it always takes the first value. So it's better to avoid it. If you want to keep a copy of an old value, change the line into a comment line.

Editing input files might appear a little old-fashioned today, but has the big advantage of easy documentation and allows to run and control the program from scripts, which can be very effective e.g in optimization procedures.

1. Example: Selecting a Magnetic Field Model

After

. $WAVE/shell/set_wave_environment.sh

our starting point is ***$WAVE/input/wave.examples*** . We copy it to ***wave.in***:

cp $WAVE/input/wave.example wave.in

and enter

WAVESHOP

The GUI pops up, select in the upper menu bar ***Comment*** and to change the user title

WAVE.EXAMPLE

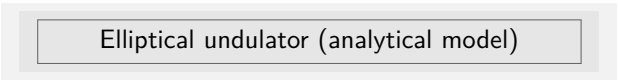
to

Planar Undulator

Hit ***OK*** to leave the widget. Select ***Sel. Mag.***.

A menu pops up:

– Click



to deselect the current model and

to select the planar undulator model.

Now we set up an undulator model with ***99*** main poles and two endpoles, i.e. a ***50*** periods undulator. The period length is ***0.033m***. In addition, the third harmonic should be at ***500eV*** (Fig.5).

Note:

- Selecting a harmonic overrides the magnetic field value given in the shown menu.
- For this model, ***z*** is the longitudinal coordinate, while the WAVE coordinate system is such, that ***x*** is the longitudinale axis.

We close the last two menus and go back the main input menu.

Next, we adjust the observation pinhole.

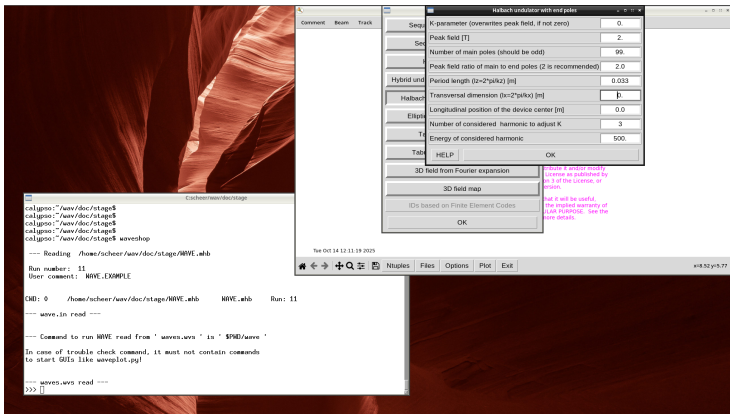
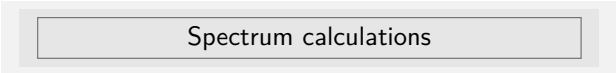
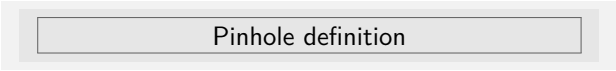


Fig. 5: Parameters of the undulator

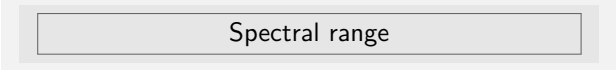
– Click





and set the pinhole width and height to ***0.002***

– Hit ***OK*** and choose:



Set the lowest and highest photon energies to ***495.0*** and ***505.0*** eV, respectively.

We run WAVE by clicking ***Run*** in the toolbar.

In the terminal we see some warnings, which can be ignored. They could be relevant in the case of very precise calculations. To avoid them, one could increase the number of grid points in the Pinhole menu from 21 to 51, but that slows down everything for nearly no benefit.

Click the button ***Plot*** , select ***Overview*** , and check the results.

Before we finish this examples, we open the output file ***wave.out*** with a text editor.

This file contains a summary of the input variables and results of the calculations, as well as warnings and error messages. At the top you see your user comment (the variable ***CODE*** in ***wave.in***) and the current run number, which is stored in the file ***WAVE_CODE.DAT***. The output file is a good place to check the input parameters of the magnetic field model, the beam parameters etc..

As a beginner, you should at least once study the file thorougly, although it might contain some information you don't understand at this moment. . .

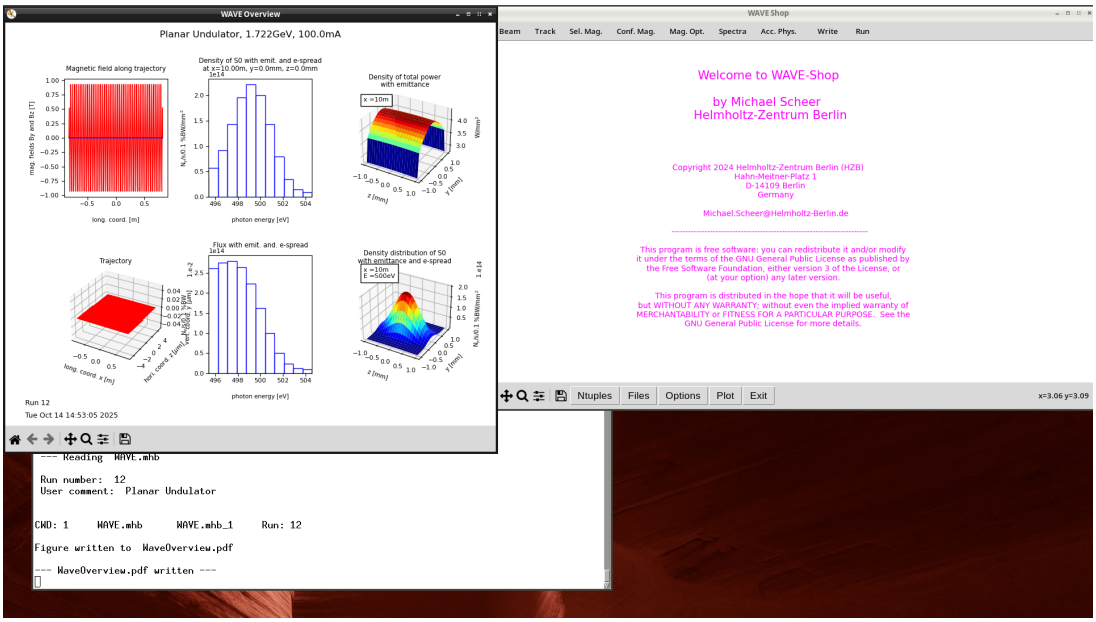


Fig. 6: Overview of plots

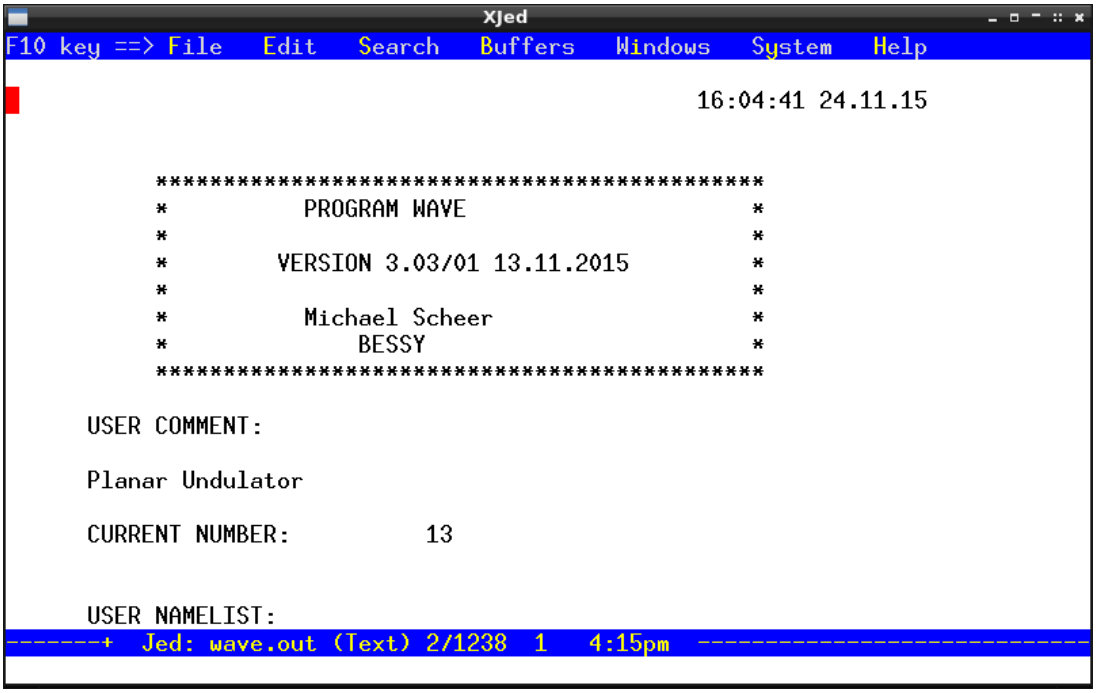


Fig. 7: The output file *wave.out*

Now it's up to you, to fumble around with the parameters of the menus described above and watch the effects. In case of trouble, you can always go back by copying ***$WAVE/input/wave_example_1.in*** to ***wave.in*** .

Proposed exercise: Compare ***wave.in*** and ***$WAVE/input/wave.examples*** .

2. Example: Tracking through Magnetic Fields

This examples illustrates the tracking mechanism of WAVE (and by the way the use of the command line... for the plotting.) After

. $WAVE/shell/set_wave_environment.sh

We start from *$WAVE/input/wave.examples* and use the python script *../python/wave_example_2.py* to create *wave.in*. But before, we have a look on it. We open *../python/wave_example_2.py* with an text editor (Fig. 8):

```
ReplList = [ \
["CODE='WAVE.EXAMPLE'","CODE='Example 2: Tracking through Magnetic Fields'"], \
["BYGOFF=0.0","BYGOFF=0.00005"], \
["KELLIP=1","KELLIP=0"], \
["KHALBASY=0","KHALBASY=1"], \
["B0HALBASY=2.","B0HALBASY=1."], \
["PKHALBASY=1","PKHALBASY=0"], \
["AHWPOL=21.","AHWPOL=99."], \
["ISPEC=1","ISPEC=0"], \
]
```

Fig. 8: First part of the script to set-up *wave.in* for the second example *wave.out*

The list *ReplList* contains the strings to be replaced in *wave.in*. Each line contains two strings. The first string in *wave.in* is replaced by the second. Most of the variables you known from example 1, but the variable *ISPEC* belonging to the namelist *$CONTRL* is new. Changing *ISPEC=1* to *ISPEC=0* , switches the spectrum calculations off. Also new is the variable *BYGOFF* , which is a global field offset added to the undulator field, e.g. to simulate the earth field.

Now we execute the script, copy the resulting file, and run WAVE :

```
python3 ../python/wave_example_2.py
cp ../input/wave_example_2.in wave.in
wave
```

On the terminal, two warnings appear, indicating that the trajectory is not correctly adjusted. This means, a final kick and a final offset remain. We start the plotting GUI:

```
wplot
```

Click the button *Plot* , select *Overview* , and check the results.

We see in the overview Fig. 9 that the trajectory is distorted as expected due to the external field,

Sometimes, it's useful to adjust the trajectory, e.g. if you want to have a symmetric one with respect to the center of the straight section or if for a given field model the correct endpole constellation is missing. This can be achieved with by tuning the electron parameters in the namelist *$CONTRL*:

The most interesting variable is *XINTER* , which defines the x-coordinate the other variables like *YSTART* or *VZIN* refer to.

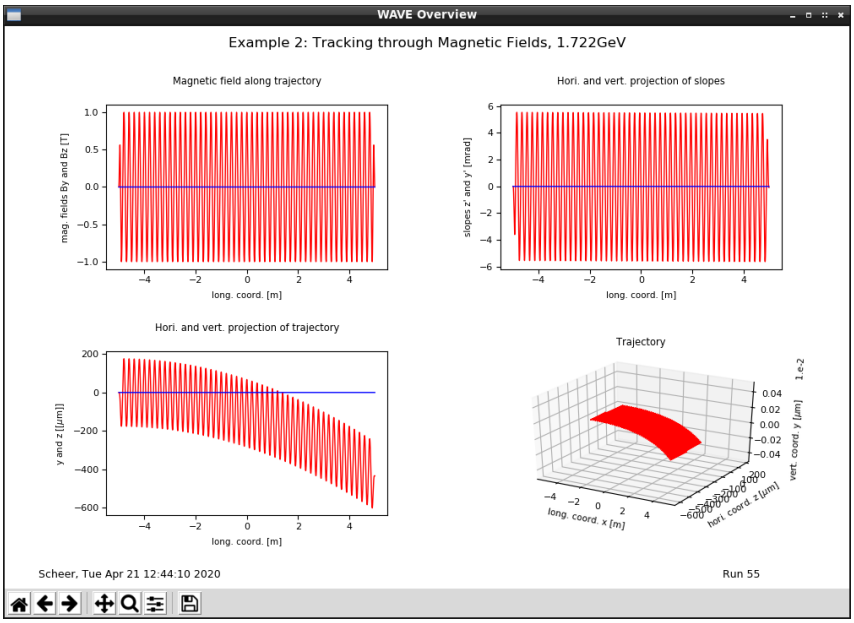


Fig. 9: The trajectory with ***XINTER=-9999.***

Edit ***wave.in***and set

XINTER=0.

Then we save the file and enter:

wave
wplot
nz()

and see (Fig. 11) a trajectory, where the electron passes the origin with zero slope according to:

YSTART=0.
ZSTART=0.
VXIN=1.
VYIN=0.
VZIN=0.

at

XINTER=0.

Thus, if ***XINTER*** is not ***-9999.*** , WAVE tracks first the electron from ***X = XINTER*** back to ***XSTART*** and then starts the actual tracking through the magnetic field.

You can save the picture with the corresponding toolbar button or typing "s" when the cursor is over the plotting window or by typing e.g.

pplot("trajectory.png")

or

pplot("trajectory.pdf")

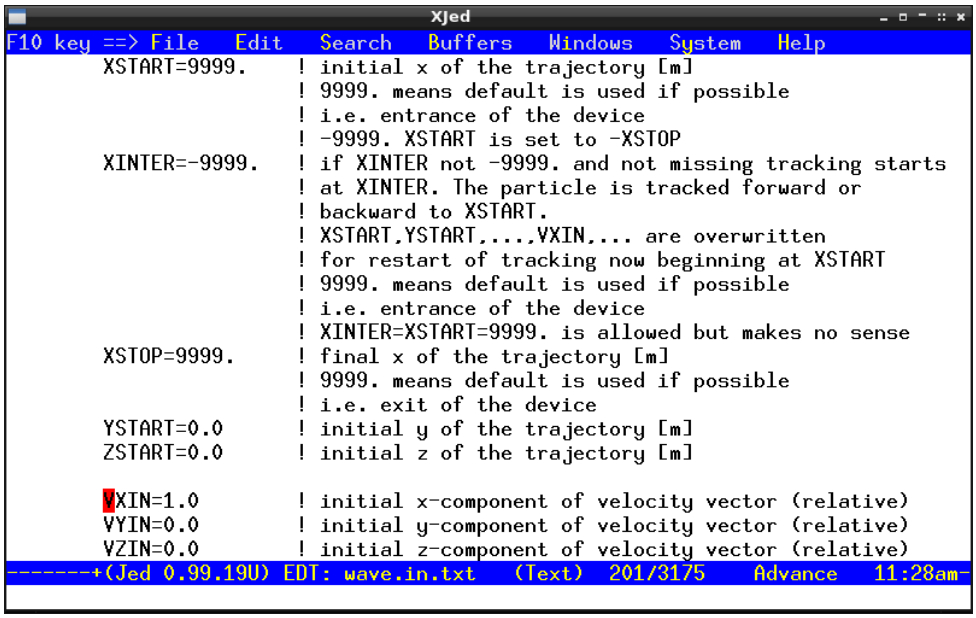


Fig. 10: Electron variables

in the Python shell. (If the overview is plotted a file *WaveOverview.pdf* is written automatically.) The last plotting command

nz()

can of course also be triggered from the GUI menu. The "n" referes to the fact, that the data stem from an n-tuple. You can also try

nby()

and so on. You might have alread noticed, that the plotting commands executed by the GUI are echoed in the Python shell. The command echoes show you the syntax and parameters of the plotting commands applied. So you can learn short-cuts for plotting from the Python shell.

If you don't see these command echos, the configuration file *waveplot.cfg* is missing in your working directory. You find an example in the input-directory *$WAVE/input* .

If *waveplot.cfg* is present, plotting options are read from this file. E.g. you can set the window size, trigger saving of all plots or writing plotted data to ASCII-Files. To see all options, check this file.

Some options can also be toggled in the GUI or by commands, e.g.

optecho(1)

or

optecho(0)

or

optstat(1)

or

optstat(0).

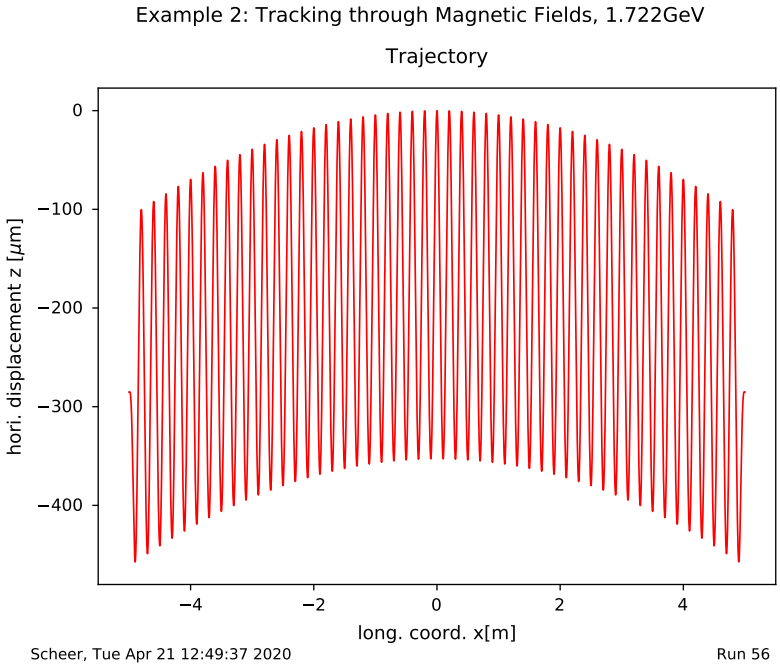


Fig. 11: Adjusted trajectory with ***XINTER=0.*** .

Note: If you calculate synchrotron radiation from this trajectory, i.e. ***ISPEC=1*** , the back tracking at the beginning does, of course, not contribute to the radiation or irradiated power of the device. By the way, this power is given in the output file ***wave.out*** . The power is normalized to the beam current ***DMYCURR*** in the namelist ***$CONTRL*** . In ***wave.out*** you also find the energy loss of the electron having passed the magnetic field. This is controlled by ***IENELOSS*** also namelist ***$CONTRL*** . For very long undulators or wigglers, the energy loss influcences the trajectory and the spectrum of synchrotron radiation. For a single, typical insertion devices, this can be neglected, i.e. ***IENELOSS=0*** can be used.

3. Example: The Concept of Source Points

Synchroton radiation is mainly emitted into a narrow cone on the scale of $1/\gamma$. Consider a wave-length shifter (WLS) with a main pole and two endpoles. Such a device has a deflection angle that is usually much larger than $1/\gamma$. Therefore, an observer at point P sees up to three sources of radiation from the WLS, namely the points of the trajectory, where the electron points to P as sketched in Fig. 12. These points, we call source points. Actually, they are not really points, but pieces of the trajectory due to the finite opening angle of the radiation cone.

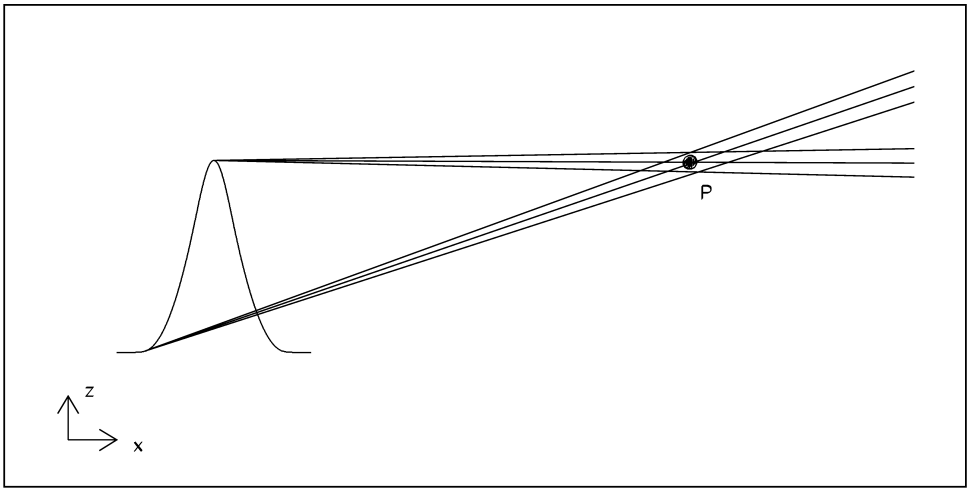


Fig. 12: Source points of a wave-length shifter for observation point P

Let's have a closer look. Enter in the terminal:

```
. $WAVE/shell/set_wave_environment.sh
cp ../input/wave.example wave.in
waveshop
```

– The GUI pops up, select in the upper menu bar ***Comment*** and change the user title

User title	Example 3: The Concept of Source Points

– Click ***Track*** in the toolbar
and set

number of steps per meter	100000

to increase the precision of the spectrum calculations in the undulator mode. Hit ***OK***.

– Click ***Sel. Mag.*** in the toolbar and in the menu

Elliptical undulator

Halbach undulator/wiggler with end poles

and set the peak field to ***6.0T*** , the number of main poles to ***1*** , the period length ***0.7m*** . Hit *OK*, again *OK*.

– Click ***Spectra*** in the toolbar and in the menu

Pinhole definition

single point

Click *OK*.

– Click

Spectral range

and set the lowest photon enery to ***50.*** , the highest to ***25000.*** , and the number of photon energies to ***2001*** , then hit *OK* (Fig. 13).

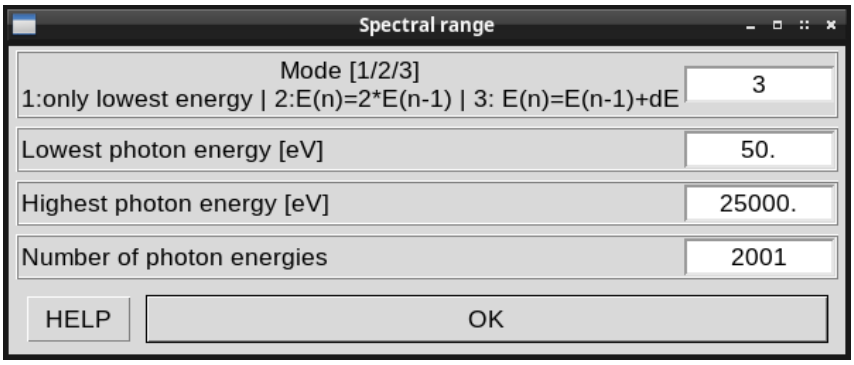


Fig. 13: Setting of the spectral range

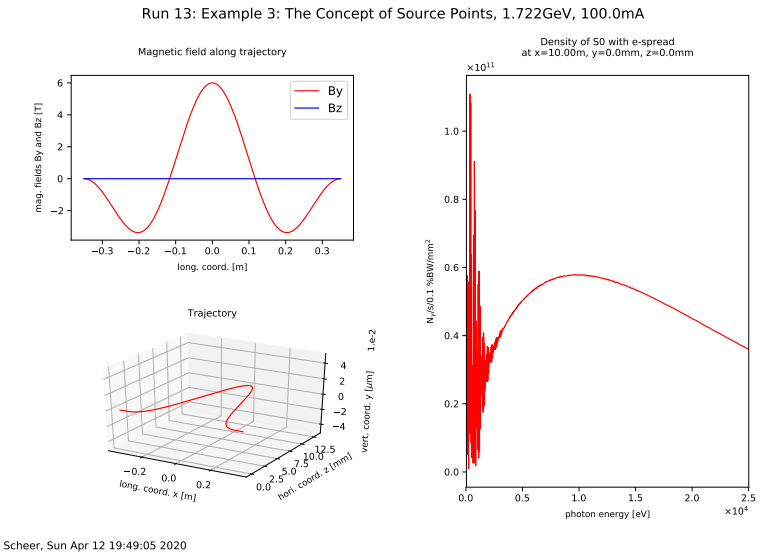


Fig. 14: The spectrum of an WLS calculated in the undulator mode

Close the menu and click ***Run*** in the toolbar and plot the overview.

The spectrum (right plot in the overview Fig. 14) looks a little weird. We leave the GUI, edit ***wave.in*** and set

FREQLOW=2000.
FREQHIG=2010.
NINTFREQ=201
IEFOLD=3

To run WAVE , enter:

wave

When WAVE has finished, enter

wplot

We choose in the plotting GUI

Plot/Spectra/Flux-density/Fd

The spectrum (red) in Fig. 15 is not as smooth as we would expect from a wave-length shifter. The synchrotron radiation of a single electron is fully coherent, thus there are interferences between the radition of the different poles of the WLS. But the phase advance between the poles is so high, that the corresponding huge number of oszillations can not be resolved by the integration algorithm of WAVE . The observed spectrum is therefore a mixture of real interferences and a numerical noise depending on the integration parameters.

However, the energy-spread of a real beam smears out the spectrum. To plot the smeared spectrum goto to the plot GUI and select:

Options/Same Zone
Options/Next Color
Menu/Spectra/Flux-density/Fd with e-spread

The resulting spectrum (blue line) is smooth as would be expected from a wave-length shifter.

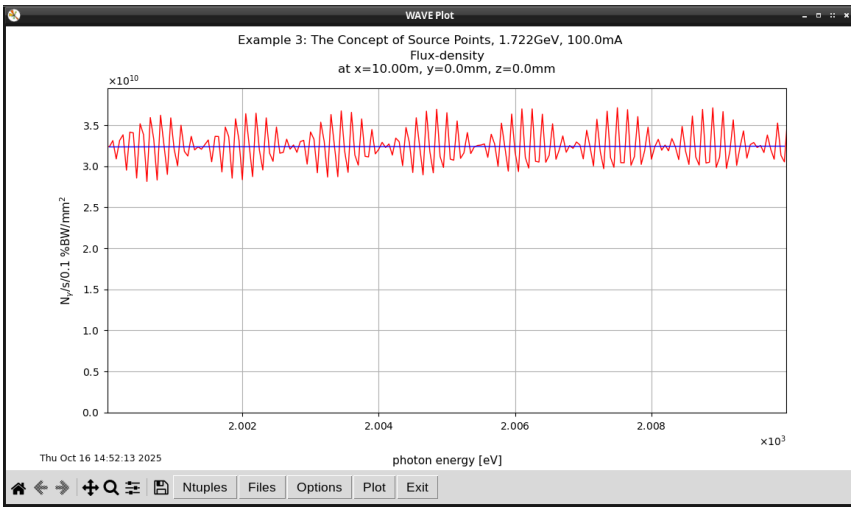
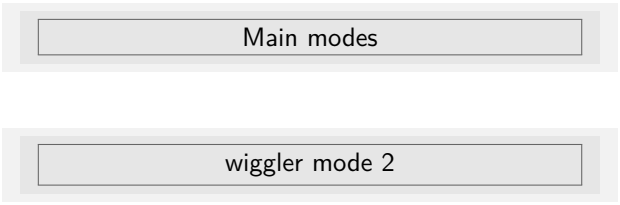


Fig. 15: The spectrum of an WLS calculated in the undulator mode. Red: Without e-spread, blue: with e-spread.

Terminate the plotting and start the GUI ***WAVESHOP***.

In the toolbar hit ***Spectra*** and select in the menu "Spectrum calculations"



Hit ***OK*** and run WAVE .

To plot the spectrum including the energy-spread, we choose in the plotting GUI

Menu/Spectra/Flux-density/Fd with e-spread

or enter in the Python shell

hcfluxden("fde")

Fig. 16 shows the same result, as in the previous run with the undulator mode. We conclude, that the interferences between the poles of the WLS are real, but can not be calculated correctly and can not be observed for real beams with a finite energy-spread. Hence, it is justified to neglect them,

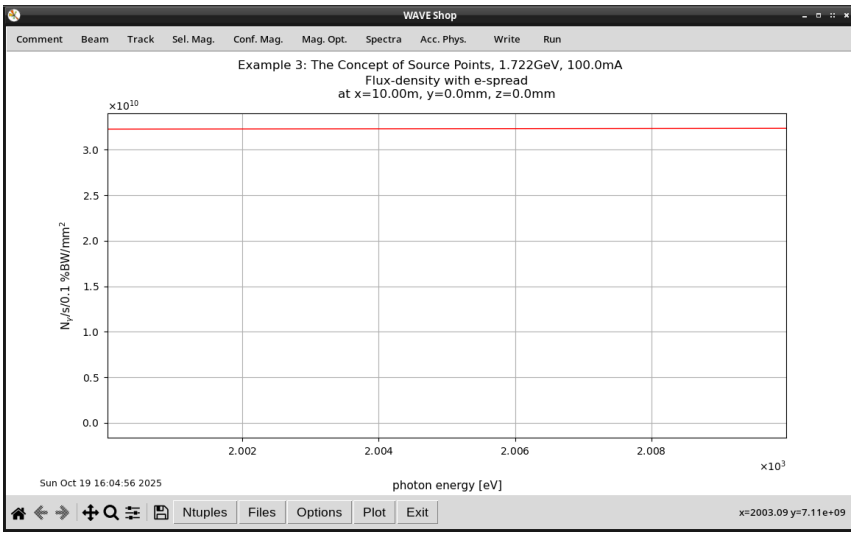


Fig. 16: The spectrum of a WLS calculated in the wiggler mode including the energy-spread of the beam

i.e. to calculate the radiation from each pole seperately and to sum up the contribution of all poles incoherently. This is what the wiggler mode of WAVE does. The decision, if the wiggler mode should be applied (and especially together with Schwinger's formular) depends on the regime, where the spectrum is calculated. For a wiggler, this means the undulator mode is needed in the spectral range of the lower harmonics. In doubt, try the different modes and have a look on the results.

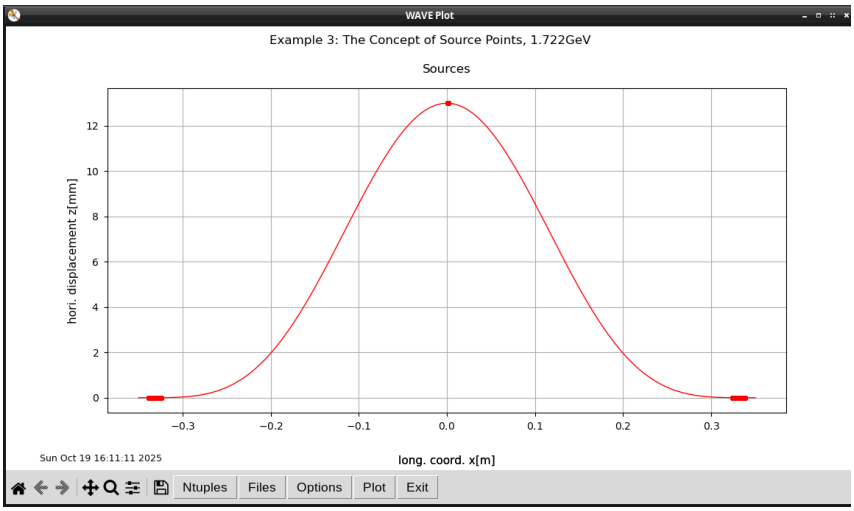


Fig. 17: Source points of a WLS

In the plotting GUI, choose:

Menu / Fields and Trajectory / Sources / z(x)

As shown in Fig. 17, WAVE has found three source points. How does it work? WAVE tracks first the electron and stores the trajectory. Then it checks for all steps of the trajectory, if a beam with

a given opening angle passes at least partically a collimator. The first step satisfying this, is marked as the beginning of a source, the last as the end.

This procedure and the set-up of the collimator are controlled by the variables of the namelist ***$COLLIN***. The main modes of WAVE namely the undulator and wiggler modes set default values for these variables, which should work in most cases. In addition, the main modes control the intergration method ***ISPECMODE*** and the number of integration steps ***NLPOI*** used to calculate the spectrum.

Proposed exercise:

Try the expert mode, i.e. ***IUNDULATOR=0, IWIGGLER=0*** and play with the variables ***WGWINFC, NLPOI*** and the variables of the namelist ***$COLLIN***.

4. Example: APPLE-II Undulator

The most realistic undulator model [1] of WAVE is an undulator build of permanent magnet blocks (rare earth componds like NdFeB, shortly REC). The field of these blocks is calculated with the current sheet method. Due to the large number of blocks of an undulator the field calculations are rather slow, but the field is very realistic, and an undulator can be modelled with all its features like gap and shift changes.

The namelist *$REC* holds the variables of the undulator. It also includes gap taper and field errors. To set up the input file ***wave.in***, enter:

```
. $WAVE/shell/set_wave_environment.sh
python3 ../python/wave_example_4.py
cp $WAVE/input/wave_example_4.in wave.in
```

After the WAVE run, you'll find a file named ***rec_plotm.eps*** , which shows the set-up of the undulator (Fig. 18).

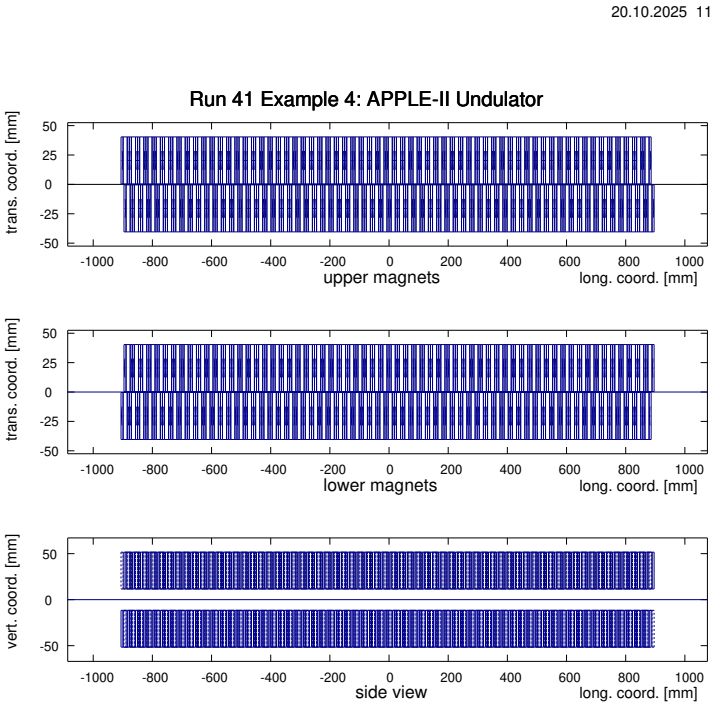


Fig. 18: REC model of an undulator

The figur shows the upper and lower rows of magnets as top views and a side view of both. To see details we click ***Conf. Mag.*** in the toolbar of *WAVESHOP* and

[1] See also example 11. for an even better model by interfacing to UNDUMAG. UNDUMAG allows to take into account the interaction between the neighbouring magnets, while WAVE only consideres static magnets with $\mu = 1$. This interaction might affect especially the first and second integrals, i.e. the kick and off set at the end of the trajectory.

REC-undulator (Option KBREC)

Options

Plotting options

and set the plotting range

RPLXMN = -920.

RPLXMX = -850.

and rerun WAVE .

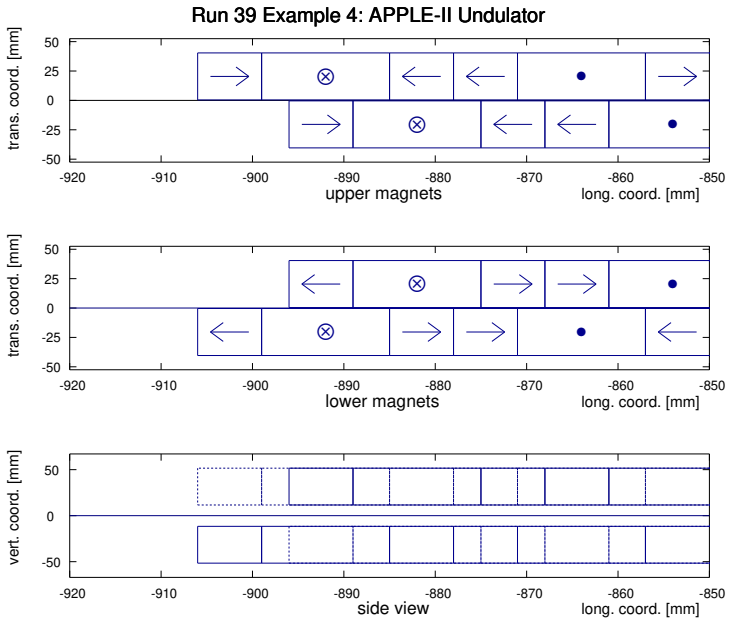


Fig. 19: Endpoles of the REC model

Now, you see the details of the endpoles, the shift, and the magnetization of the blocks. More complex situtation can be handle by an input file containing arrays of magnet. This will be explained in one of the following examples.

The overview of this run is shown in Fig. 20. It shows the elliptical trajectory and the corresponding first harmonic.

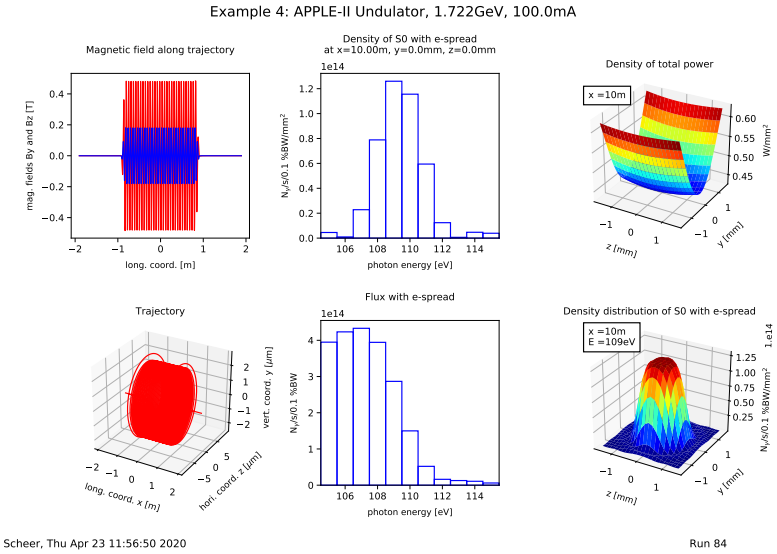


Fig. 20: Example of an APPLE-II undulator

Proposed exercise:

In the option menu of the REC-Undulators is a menu to define errors of the magnetic blocks. Give it a try, and check also the magnetic field errors option in the menu of the global magnetic field options.

5. Example: Simple Build-In Optimization

A very common situation is, that you want to tune input parameters with respect to a figure of merit. WAVE has build in two simple procedures to accomplish that. The first is an iterativ estimation for the case of one parameter to adjust, the second a Monte-Carlo for more than one parameter.

Consider example 4For the given gap, we want to find the shift such, that the vertical and horizontal field of the undulator are the same, i.e. the device acts as an circular undulator.

In a terminal, we enter:

```
. $WAVE/shell/set_wave_environment.sh
python3 ../python/wave_example_5.py
cp ../input/wave_example_5.in wave.in
wave
```

We see in ***wave.out*** , that the max. vertical and horizontal fields ***BYMAX*** and ***BZMAX*** differ. Now, we adjust the shift parameter of the undulator. Enter:

```
cp ../input/wave_example_5.adjust wave.adjust
cp ../python/wave_adjust_example_5.py wave_adjust.py
```

Have a look on ***wave.adjust*** (Fig.22). When a file like this is passed as an argument to WAVE, WAVE will adjust the parameter ***URSHIFT(1)*** of ***wave.in*** by a Monte-Carlo search. Of course, this also works with other filenames and variables of ***wave.in***. During the optimization procedure WAVE executes the script ***wave_adjust.sh*** , which calls another instance of WAVE and ***wave_adjust.py***.

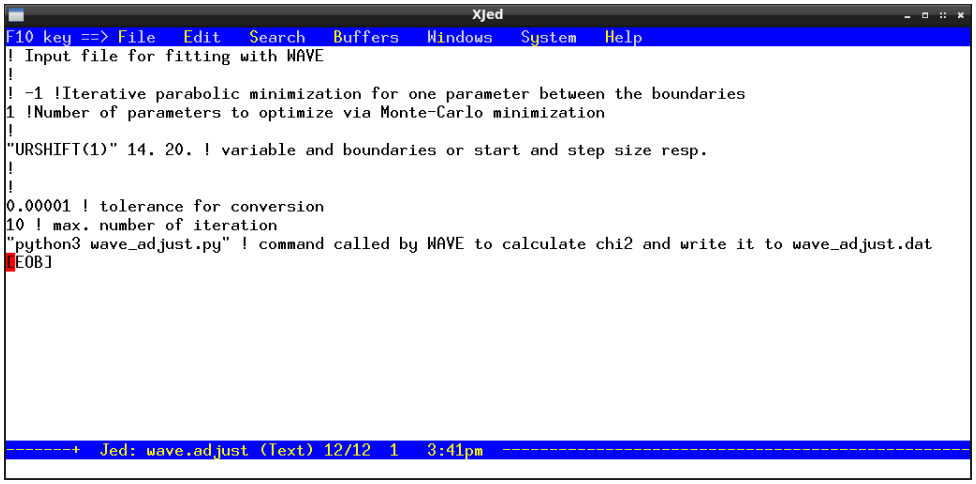


Fig. 21: Input file for the optimization procedure

After the run of this instance, the command given in the last line of ***wave.adjust*** is executed by the master process. In our example, this is the python script ***wave_adjust.py*** . What does this script

(Fig.22) do?

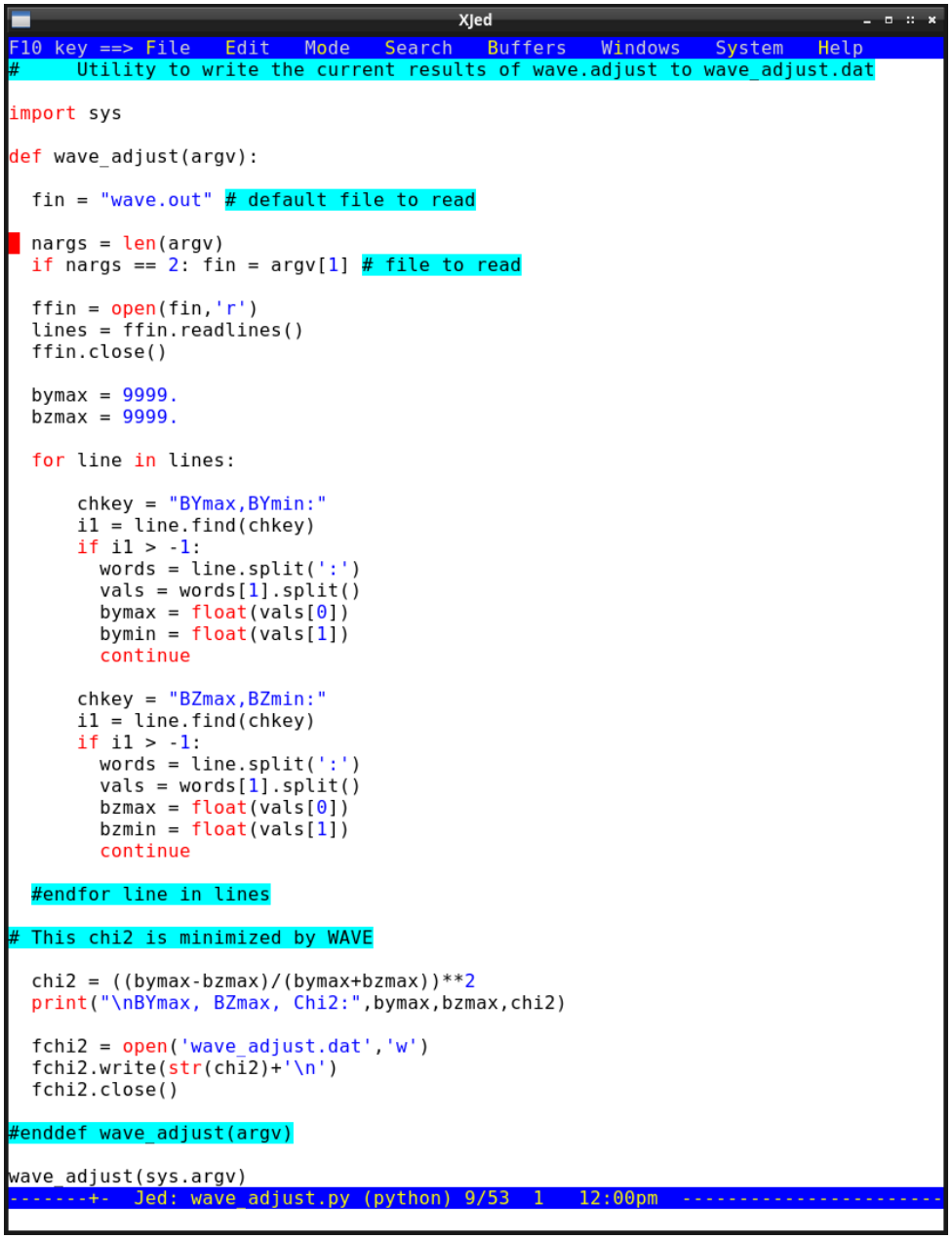


Fig. 22: Example of a program to calculate the figure of merit

It opens ***wave.out*** , and looks for the lines containing the keywords like ***BYmin***. When a matching line is found, the program reads from the line the values of ***By*** or ***Bz***, respectively, and calculates the χ^2 to be minimized. The resulting χ^2 is written to ***wave_adjust.dat*** and evaluated by the master process.

Let's try. Enter:

```
cp wave.in wave_adjust.in
wave wave.adjust
```

WAVE loops and writes ***wave_adjust.out*** . The data in this file show a minimum for the shift of about ***19mm*** (Fig.23). This minimum value found is also used to re-run WAVE.

Start the plotting GUI by entering:

```
wplot
```

and enter in the python shell:

```
adj = ncread("adj","sh:chi2","wave_adjust.out")
nplot(adj,"sh:chi2")
txyz("wave_adjust.out","Shift[mm]","Chi2")
```

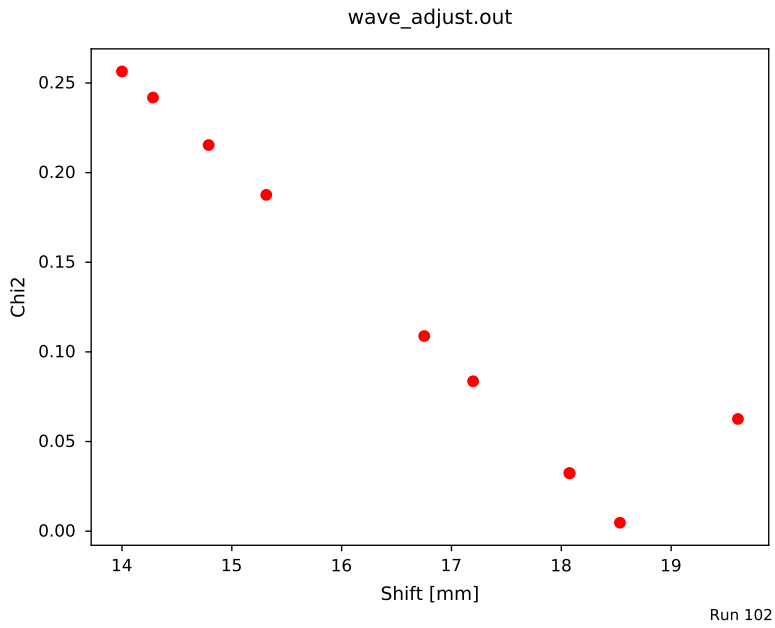


Fig. 23: Random variation of URSHIFT(1)

The plot you get, might differ from that shown here, due to different seeds of the Monte-Carlo search.

To get a precise value, we edit ***wave.adjust*** , and set the number of parameters to ***-1*** and the boundaries to ***18.0*** and ***20.0*** . Then we repeat

```
cp wave.in wave_adjust.in
wave wave.adjust
```

and find now in ***wave.in*** the best value of about

URSHIFT(1)= 18.6

Since we have required a precision for the minimum $\chi^2 \leq 10^{-6}$, the ratio of the vertical and horizontal field now agrees on the scale of $\sqrt{10^{-6}}$.

The effort in this example might appear quite high for such a simple probleme, which could be solve sufficently by a quick interpolation. However, it shows the usage of scripts and the advantage of command line driven optimization procedures.

6. Example: "Real" Beams, Part I

Up to now, we have only considered single electron beams, except in example 3, where we have seen the effect of the energy-spread of the beam. In this example, we learn about the first of two methods how WAVE takes the effects of real beams, i.e. emittance and enery-spread into account.

In modern low emittance storage rings the spatial and spectral distribution of the synchrotron radiation does not vary very much for the different electrons in the beam phase-space. Thus, the shape of the radiation cone is more or less the same, it is only shifted according to the position and slope of the considered electron. Therefore, it is possible to take the beam emittance into account by folding the radiation cone of the reference electron with a distribution that reflects the phase-space. This distribution is usually a 2D-Gaussian with

$$\begin{aligned} \Sigma_y^2 &= \sigma_y^2 + d^2 \times \sigma_y'^2 \\ \Sigma_z^2 &= \sigma_x^2 + d^2 \times \sigma_x'^2 \end{aligned}$$

where σ_y, σ_y', σ_z, σ_z' denote the vertical and horizontal sizes and divergences of the beam and d the distance between the source and the observation plane. Enter:

```
. $WAVE/shell/set_wave_environment.sh
python3 ../python/wave_example_6.py
cp ../input/wave_example_6.in wave.in
wave
wplot
```

Use the plotting GUI to produce the Fig. 24 or enter in the python shell:

```
zone(1,2,1)
ndistpinh()
nextzone()
ndistpinv()
```

If you want to change parameters for varies runs, but plot always the same, you can put these lines into a file

waveplot_startup.py

The commands in this file are exceuted at the start of ***wplot*** , until the end of the file is reached, or a line with a key-word ***EOF*** is found. Of course, this can also handle the plotting zones by using the ***Options*** menu of ***wplot*** .

The mean value of the histgrams agree with the position of the observation pinhole as given in ***wave.out*** . We see, that the position of the pinhole is shifted. This has been achieved by setting

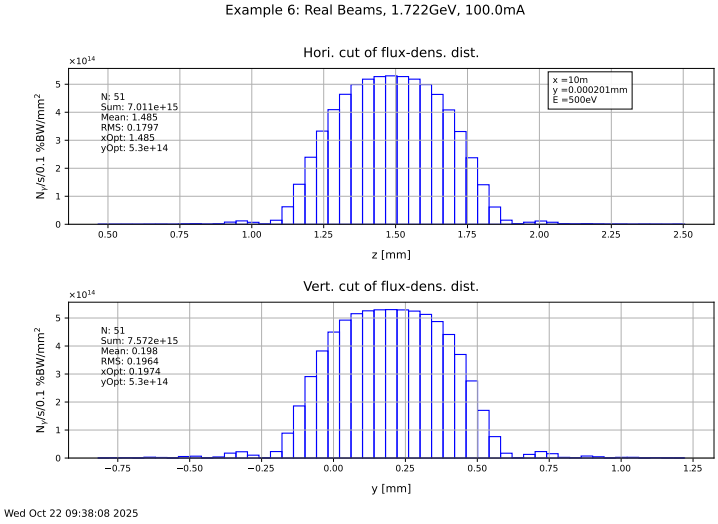


Fig. 24: Horizontal and vertical cuts through the flux-density distribution

YSTART=56.1e-6
ZSTART=648.e-6
VYIN=12.87e-6
VZIN=74.4e-6
PINCEN(2)=-9999.
PINCEN(3)=-9999.

in ***wave.in***. The values for the start conditons of the electron correspond to three sigmas of the beam sizes and divergences as given in the namelist ***$WFOLDN***. The values for the center of the pinhole trigger the shift of the pinhole, as is explained in the namelist ***$PINHOLE***.

When we set in ***wave.in***

YSTART=0.0
YSTART=0.0
ZSTART=0.0
VYIN=0.0
VZIN=0.0

as usual, we get nearly the same histograms, except that they centered on-axis. Hence, the ansatz of the folding procedure is verified.

A similar folding procedure is applied for the spectrum to take into account the energy-spread of the beam (***ESPREAD*** in the namelist ***$WFOLDN***). This procedure makes use of the fact, that the harmonics of a undulator depend on the square of the beam energy E_{beam}. Thus, the shift of the spectrum is in first order given by

$$\frac{\Delta E_{phot}}{E_{phot}} = 2 \times \frac{\Delta E_{beam}}{E_{beam}}$$

while the shape stays in a good approximation the same.

Proposed exercise:

Try to verify the last statement by using WAVE .

7. Example: Global Magnetic Field Options

WAVE offers in the namelist ***$B0SCBLOBN*** some options, e.g. shifting, scaling, and symmetry operations of the magnetic field. It also offers to repeat the set-up periodically. In this example, we make use of these options in order to calculate the spontanous synchrotron radiation of a very long undulator section including phase shifters. But first, we consider a single cell of this section (Fig. 25). We enter:

```
. $WAVE/shell/set_wave_environment.sh
python3 ../python/wave_example_7.py
cp $WAVE/input/wave_example_7.in wave.in
wave
wplot
```

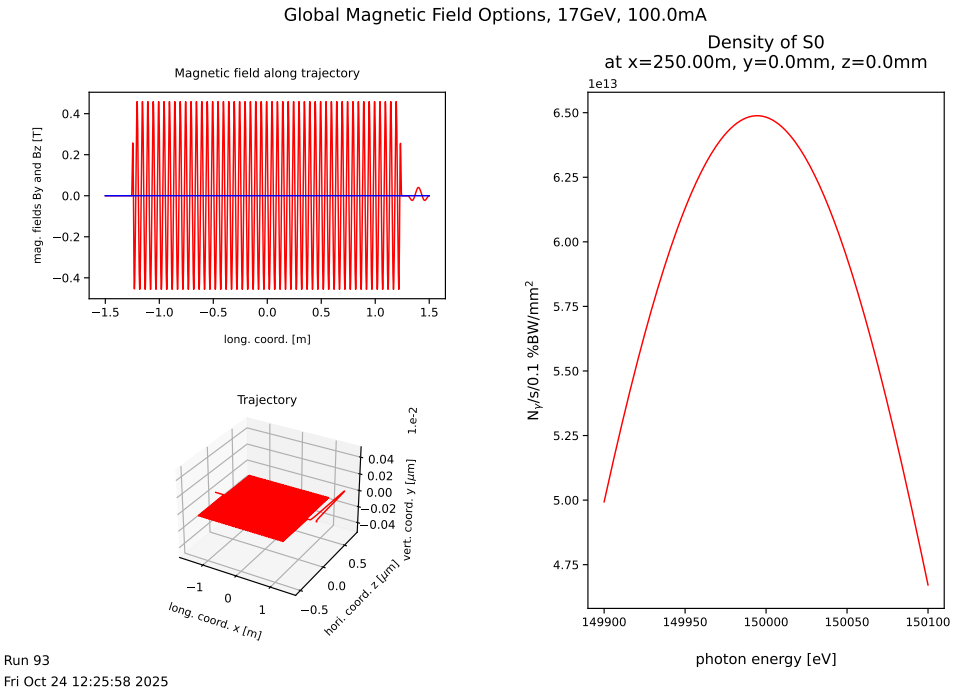


Fig. 25: Field and 9th harmonic of a single undulator section

As the overview shows, the field is a superposition a planar undulator (namelist ***$ELLIPN***) and a phase shifter (namelist ***$HALBASY***). If ***IBSUPER=1*** in ***wave.in***, WAVE accepts several field selections . The undulator parameters are set such, that the 9th harmonic is at ***150 keV*** for a beam energy of ***17 GeV*** . We now make use of a the periodic field option. Have a look on the corresponding variables in the namelist ***$B0SCGLOBN***:

```
IPERIODG=1
PERIODG=3.
PEROFFG=-1.5
```

This means, that the magnetic field is periodically repeated starting from ***X=-1.5*** with a period-length of 3 meters.

We set in ***wave.in***:

```
XSTOP=88.5
IHTRSMP=10
```

and run WAVE and get the results shown in Fig. 26

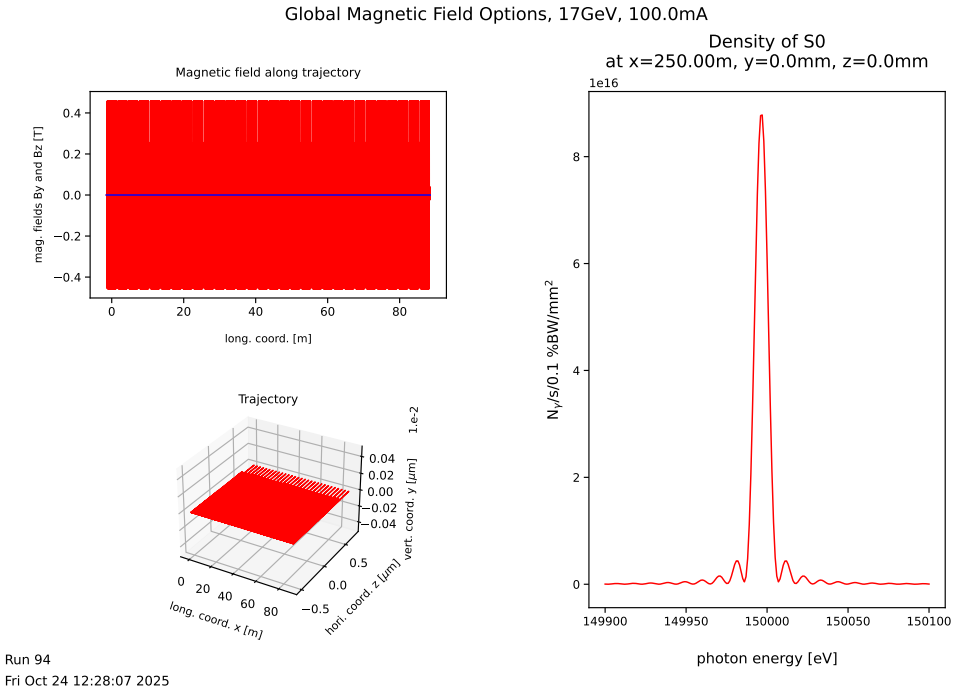


Fig. 26: Field and 9th harmonic of a long undulator section

Passing a long undulator section an electron beam loses a significant amount of energy. Hence the K-parameter of the down-stream undulators does not match the beam energy anymore. This results in a broadening of the spectrum. This can be simulated by setting

```
IENELOSS=1
```

in ***wave.in***. We run WAVE and get the results in Fig. 27.

To compensate it, we must adjust the field of each undulator accordingly. This can be achieved with the field option

```
JBMASK=100
```

and by providing a file ***wave.bmask*** , which contains a number of X-Intervalls and a scaling factor for the magnetic field for each intervall. The last ***wave.out*** gives the total and relative energy loss of the beam due to the 30 undulators. From this number we calculate the energy loss of a single undulator cell, and use the relation

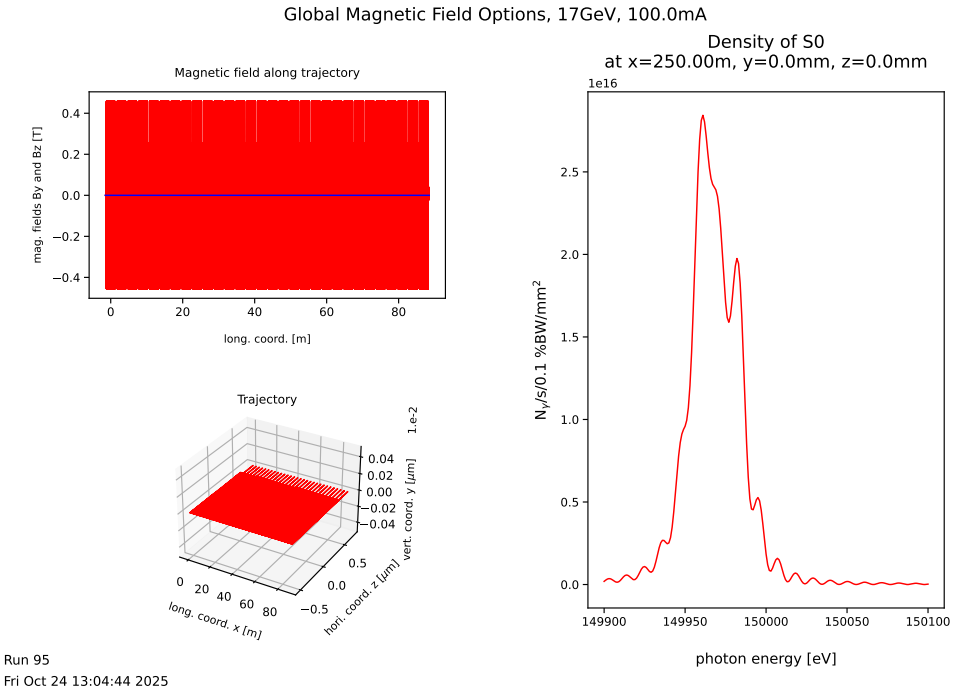


Fig. 27: Field and 9th harmonic of a long undulator section with continous energy loss of the beam

$$\frac{E_{beam}^2}{1+K^2/2} \quad = \quad const$$

to calculate the correction factor for the undulator field, as it is done in the function ***taper()*** , which also writes the file ***wave.bmask***

We use this function:

wplot
taper()
Quit()

The default values of this function correspond to this example. Beside the relative energy loss the starting K-value is taken from ***wave.out*** of the previous run as well (Look for "deflection parameter").

In ***wave.in*** we set

JBMASK=100

Please, also read the comments of the corresponding namelist ***B0SCGLOBN*** carefully and notice the difference between ***JBMASK*** and ***IBMASK***.

We run WAVE again and see in Fig. 28 that the harmonic of the undulator recovers.

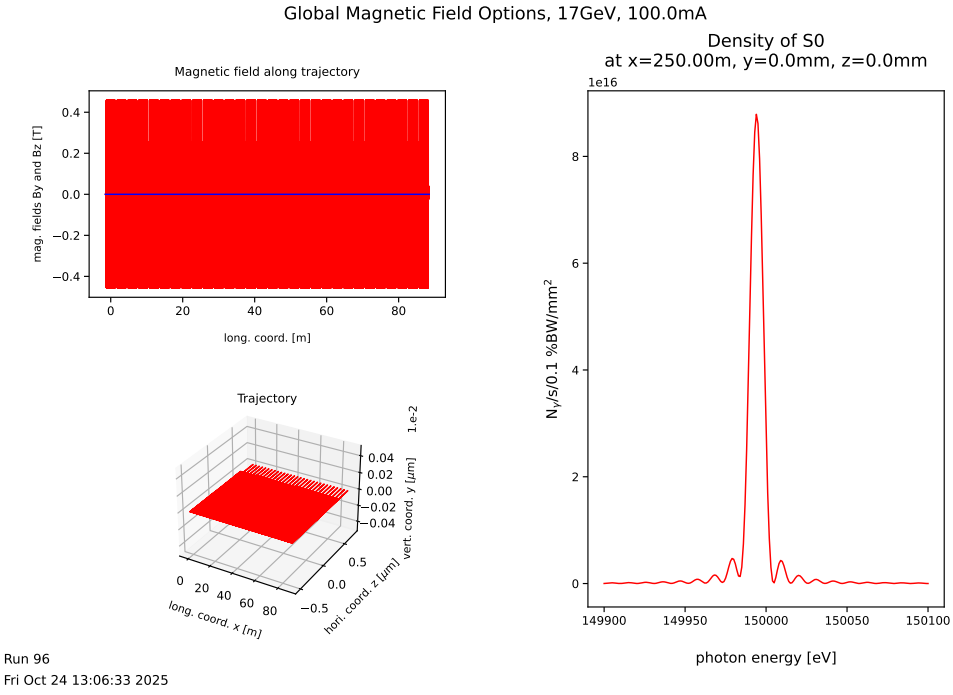


Fig. 28: Field and 9th harmonic of a long undulator section with continous energy loss of the beam and corrected undulators

8. Example: Magnetic Field Errors

The quality of an undulator depends essentially on the errors of the magnetic field. Field errors not only effect the trajectory and hence e.g. the orbit of electrons in storage rings, but also reduce the spectral brightness of the device, in particular for the higher harmonics.

WAVE offers some options to study these effects. We enter:

```
. $WAVE/shell/set_wave_environment.sh
python3 ../python/wave_example_8.py
cp $WAVE/input/wave_example_8.in wave.in
wave
wplot
```

We look at the spectrum of the flux-density. Click:

Plot / Spectra / Flux-density / Fd

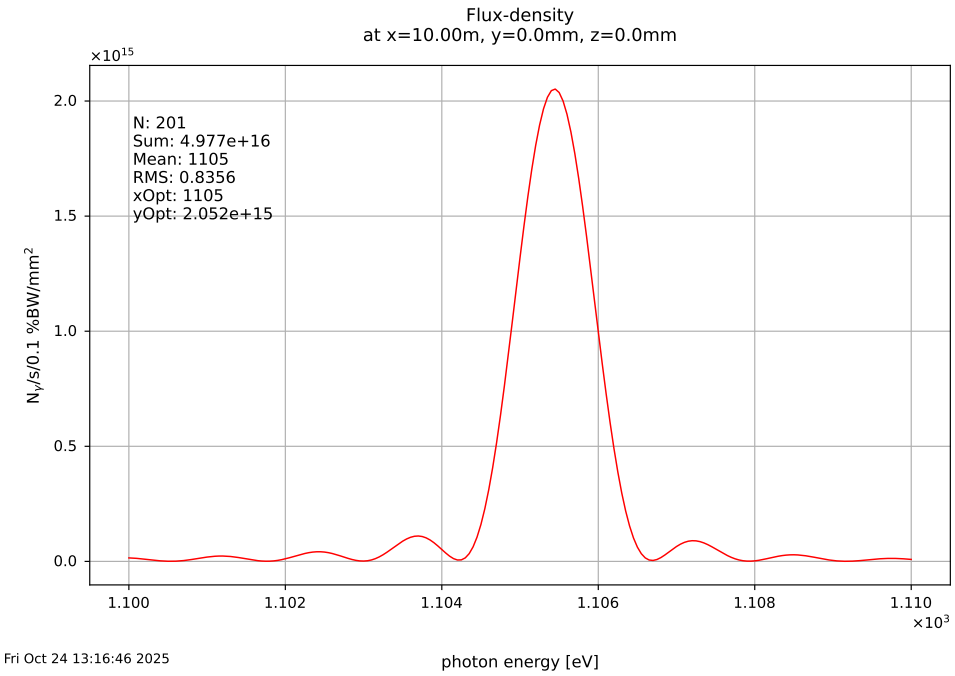


Fig. 29: Nineth harmonic of a planar undulator

The main errors of permanent magnet undulators arise from fluctuations of the magnetization of the block. This can be a variation in the strength or the direction of the blocks. In ***wave.in*** we set

```
BCRAN=0.001
UBANGERR(1)=0.2
```

In addition, feel free to modify the start of the random generator by changing ***IRNMODE*** in the namelist ***$RANDOMN***. Of course, then you will reproduce the plots of this example...

Then we run WAVE again and look at the trajectory (Fig. 30) and the spectrum by plotting the overview.

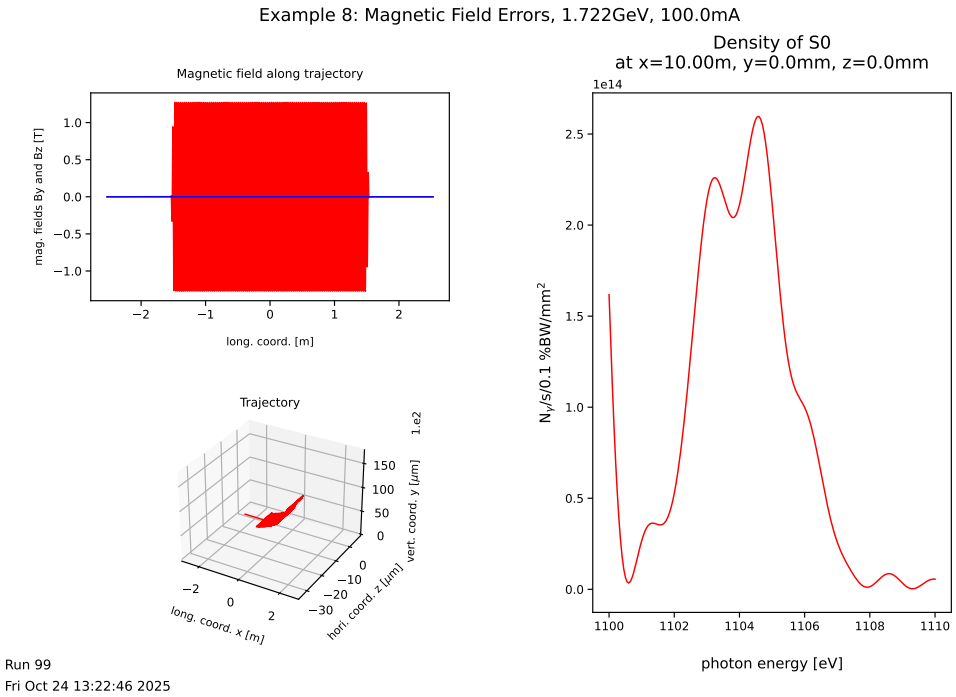


Fig. 30: Horizontal, vertical trajectory , the nineth harmonic of an undulator with field errors

The field errors result in a spoilt trajectory with a remaining kick (to see it better, keep the left mouse button pressed on the 3d trajectory plot and move the mouse to rotate the plot).

In addition the on-axis spectrum of the flux-density is ruined (Fig. 30). But the later is mainly a consequence of the stearing of the undulator, i.e. we can move the position of the observer to take this into account by looking for the point the trajectory points to:

```
OBS1Y=-8888.
OBS1Z=-8888.
```

This triggers a straight line fit of the trajectory in order to estimate the best obervation point. We run WAVE and find the spectrum of Fig. 31. The peak of the nineth harmonic is now higher; however, a significant reduction of the flux-density and a distorted shape remains due to the field errors.

We have used Gaussian errors in this example so far. Normally, one would not accept magnets with errors above a certain limit, let's say one standard deviation. This can be done by setting

```
BCRANSIG=1.0
UBANSIG(1)=1.0
```

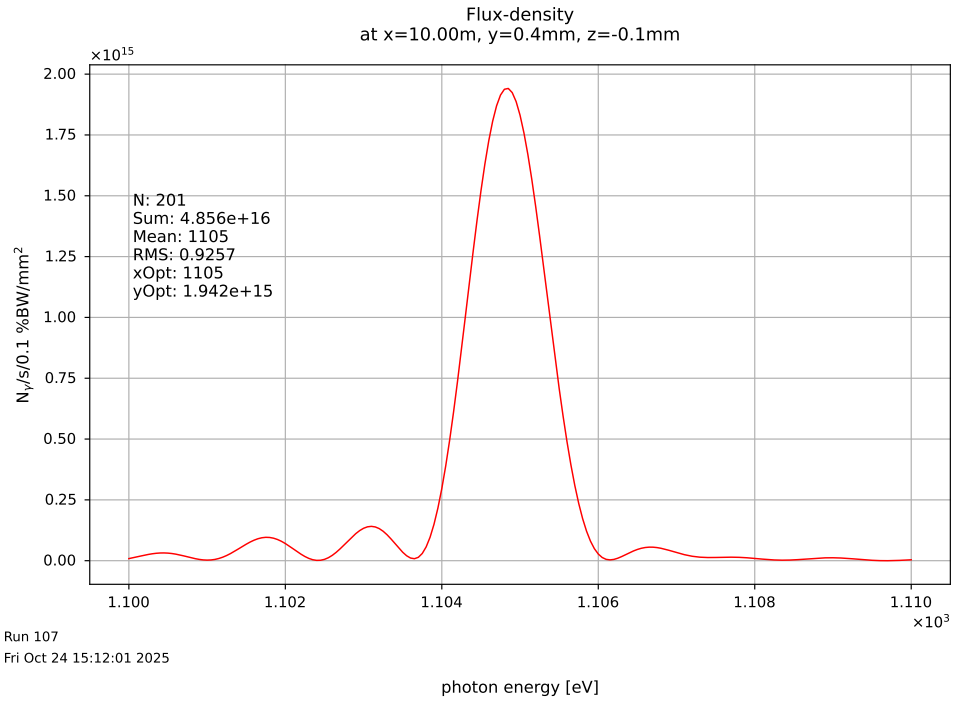


Fig. 31: Nineth harmonic at the shifted observation point

Run WAVE and enter in the plotting shell:

```
zone(2,2)
nz()
zone(2,2,3,'same')
ny()
zone(2,1,2,'same')
hcfluxden()
```

The result is shown in Fig. 32. We see a "realistic" trajectory and a good nineth harmonic.

These options allow the user to estimate the required quality of the magnets and trajectory of the undulator. There is another option ***USIGOFFY(1)***, which limits the random walk of the trajectory due to angle errors of the magnets. This simulates the shimming procedure of the undulator and also allows to estimate the required quality of the trajectory.

However, these tools are provided to produce "realistic" trajectories and give hints about the impact of field errors. They do probably not replace a thoroughly analysis of the field measurement, analysis, and shimming of an undulator.

Proposed exercises:

Check the option ***NURANMOD***. With ***NURANMOD=2***, the error ***BCRAN*** can be applied to neighboring poles. This simulates a perfect shimming of the trajectory, while keeping the phase errors and there influence on the spectrum.

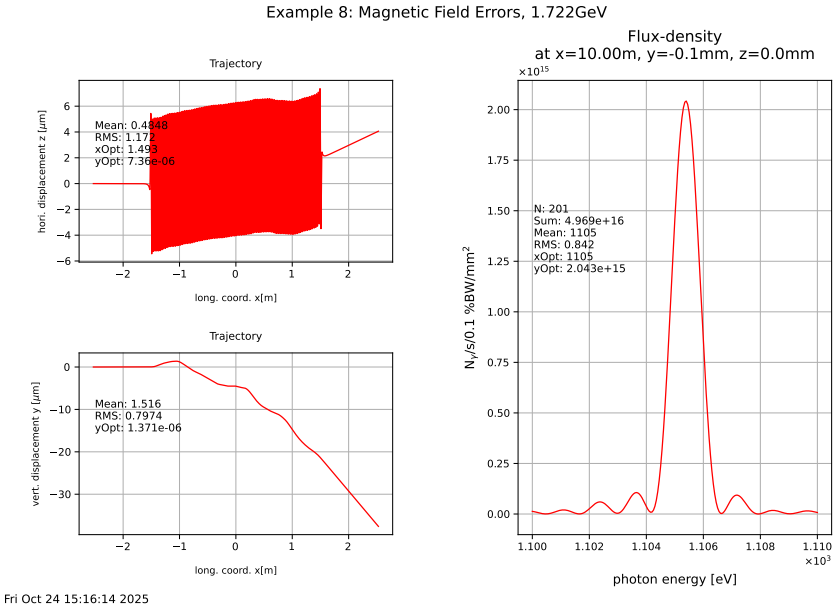


Fig. 32: Hori. and vert. trajectories and spectral flux-density of an undulator with limited errors

Dump the magnets of the model using ***IKRESTOR***, change some magnet properties, and try to use the file to investigate the effects.

Check the option ***IBERROR*** and the namelist ***$BERRORN*** and try to use it.

9. Example: Real Beams, Part II

In example 7 we've made the aquaintance of a delicate set-up. Due to the huge number of undulator periods, we are very sensitive to the input parameters of the beam. Therefore, the assumption, that the spectral and spatial distributions of the radiation is more or less the same for all electrons, is not valid anymore. Hence the folding procedures to take the energy-spread and emittance into account would fail. To overcome this difficulties, we must track many electrons according to the beam phase space distribution.

We consider a long beamline with 60 undulators. To calculate the spectrum of the spontaneous synchrotron radiation, we enter

```
. $WAVE/shell/set_wave_environment.sh
python3 ../python/wave_example_9.py
cp $WAVE/input/wave_example_9.in wave.in
cp $WAVE/input/wave_example_9_magseq.in magseq.in
cp $WAVE/input/wave_example_9_wave.bmask wave.bmask
wave
wplot
```

This run takes some minutes, since for the considered $180\,m$ long undulator section we need about a million integration steps for the spectrum calculation. For this single electron beam, we get a sharp peak for the on-axis flux-density (Fig. 33).

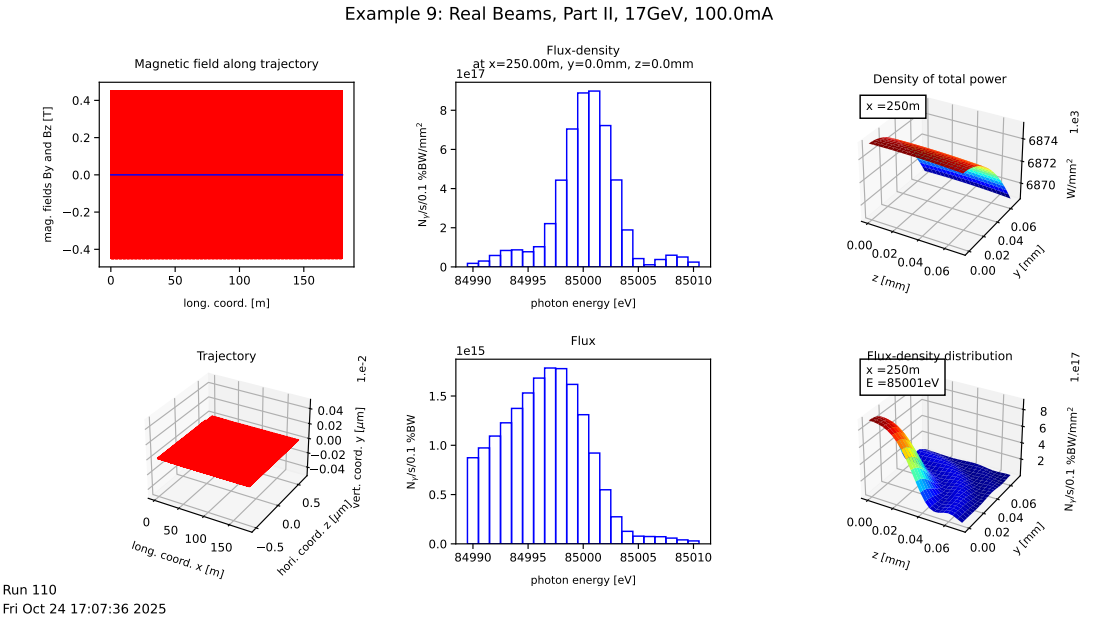


Fig. 33: 5th harmonic of a single electron beam for 30 undulator sections including continous energy-loss and tapering of the undulators

The set-up of one six meter subsection section including two undulators is given in the file ***magseq.in***. In combination with

```
XSTART=0.0
XSTOP=180.
IPERIODG=1
PERIODG=6.
```

this results in a beamline of $180\,m$ length, where the field strength is corrected for the energy loss due to the spontaneous synchrotron radiation. Each section includes phase-shifters and quadrupoles to match the phase-advance and β-functions of the 30 sections.

Note, that for time reasons, only a quarter of flux is calculated, i.e. the pinhole is shifted, such that the lower left corner is on-axis. This is achieved by

```
IPBRILL=1
PINCEN(2)=9999.
PINCEN(3)=9999.
```

Now, we come to the actual task to calculate the $5th$ harmonic for a real beam, including energy-spread, emittance, and an energy-loss with quantum fluctuations. We start step by step setting in ***wave.in***

```
IENELOSS=-1
IBUNCH=-1
NBUNCH=1000
NEINBUNCH=1
IPIN=0
```

The variable ***IBUNCH*** triggers the bunch mode, i.e. to track a given number of bunches and electrons per bunch (***NBUNCH*** and ***NEINBUNCH*** respectively). The radiation of different bunches is added incoherently, while the contributions of electrons of the same bunch are treated coherently. Thus, the setting given above calculates the radiation of 1000 electron, summed up incoherently. The resulting spectra are normalized to the beam current. Please, refer to the namelist ***BUNCHN*** for further options.

We run WAVE and get (after some minutes) the results shown in Fig. 34. We see some phase-space distributions of the beam electrons and the corresponding flux-density spectrum of a single electron beam in the lower right corner together with the same spectrum for the real beam.

We clearly see the flux-density reduced by the emittance and energy-spread. It reaches only about 0.5% of the original one.

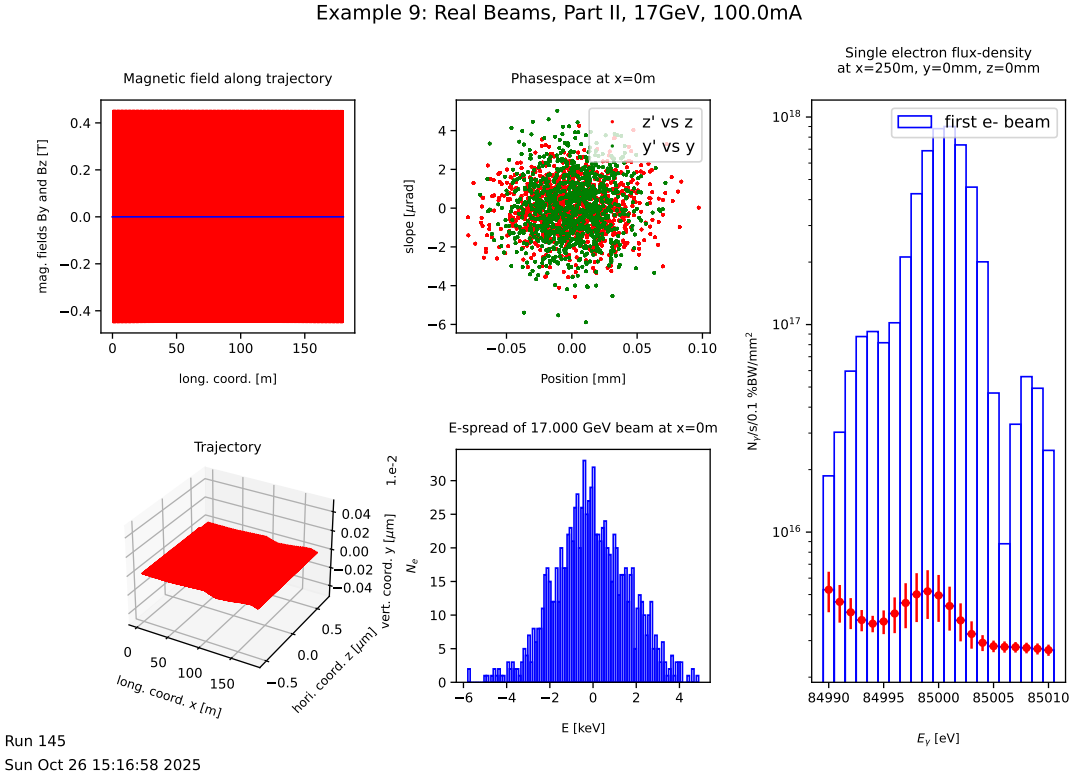


Fig. 34: Flux-density of the 5th harmonic of 1000 electrons beam for 30 undulator sections including energy-loss with quantum fluctuations and tapering of the undulators (lower right plot)

Flux through a pinhole

Since the calculation of radiation distributions in a pinhole are very time consuming, the mode ***IPIN=3*** can be used. In this mode, the radiation is not calculated for all grid points of the pinhole, but only for one randomly chosen point for each electron track. The idea is, that the random phase-space coordinates of the electron and the random observation point are not correlated, thus, for large enough a number of tracks, the flux trough the pinhole can be calculated by sampling the phase-space and the observation point simultaneously.

Edit ***wave.in*** and set:

```
IPIN=3
IPBRILL=0 PINW=0.00014
PINH=0.00014
```

and run WAVE and get Fig. 35.

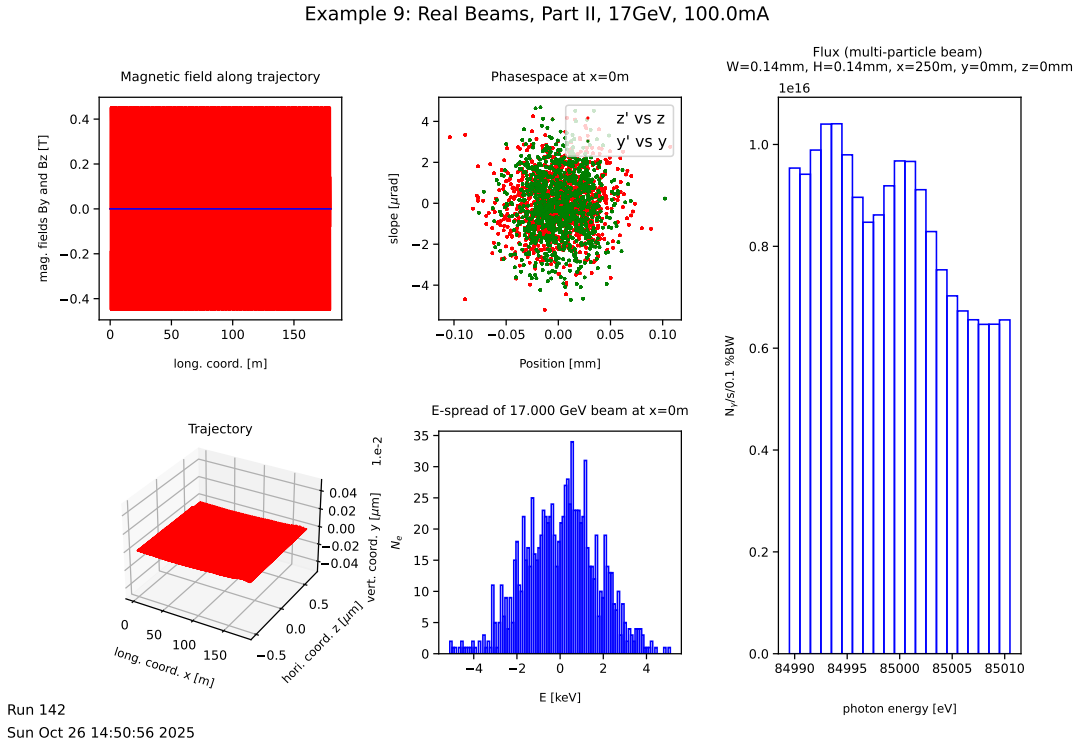


Fig. 35: Flux of the 5th harmonic of 1000 electrons beam for 30 undulator sections including energy-loss with quantum fluctuations and tapering of the undulators (lower right plot)

The flux is also significantly reduced compared to that of zero emittance and energy spread beam (Fig. 33).

Proposed exercises:

- Copy ***$WAVE/input/wave.examples*** to ***wave.in***, run WAVE , and plot the results. Store the results and try to run WAVE in the multi-particle mode to compare with the previous results.
- Check the namelist ***$BUNCHN*** in ***wave.in*** and try to provide your own phasespace distribution.
- Check the namelist ***$BUNCHN*** in ***wave.in***, and try to calculate coherent radiation of electrons.

10. Example: Parallel Processing

There are two ways to parallel processing in WAVE. The first is for the case of a single electron, i.e.

IBUNCH=0

This mode makes use of the OpenMP features of the FORTRAN compiler. To activate is set

MTHREADS= number of CPU cores to use

For

MTHREADS $<$ ***0***

all cores are used. You can explore this option e.g. by changing the setting of the 4. example accordingly.

The second way for the multi-particle mode, i.e.

IBUNCH=1

or

IBUNCH=-1

is to spawn several instances of WAVE by setting

ICLUSTER=1

If this mode is active the maximum of ***ICLUSTER*** and ***MTHREADS*** gives the number of the spawned instances. Try it by modifying the 9. example.

11. Example: Interface to UNDUMAG

UNDUMAG [2] is a code develloped at HZB/BESSY to calculate the magnetic fields of undulators and other magnetic structures.

WAVE has an interface to UNDUMAG to calculate the field of an APPLE II undulator and a planar hybrid undulator with iron poles. The parameters of the undulator and magnets are given in the namelist *$UNDUMAG*.

The files

undumag.exe

and

undumag_nam.tmp

must be installed in

$WAVE/undumag

These file names are defined by *CHUNDUMAG* and *CHUNDUNAM* in *$UNDUMAG*.

To try this example, enter:

```
. $WAVE/shell/set_wave_environment.sh
python3 ../python/wave_example_11.py
cp $WAVE/input/wave_example_11.in wave.in
wave
```

UNDUMAG writes some output files. Have a look to *undumag.log* and *undumag.beff* . There is also some graphical output, see *undumag*.eps* .

The geometry of the undulator magnet structure is shown in *undumag.eps* .

The figure 36 shows the magnetic structure and also the segementation of the magnets. Be aware, that UNDUMAG can be very memory consuming. Therefore, you must limit the number of periods and the segmentation of the magnets. These parameters can be set in the namelist *$UNDUMAG* or by using *WAVESHOP*.

The namelist *$CONTRL* holds the parameter *KBUNDUMAG*, which triggers for

KBUNDUMAG=2

the UNDUMAG run.

If you want to use the ouput of UNDUMAG instead for a subsequent WAVE run, just set

KBUNDUMAG=1

In this case, the file *undumag.wav* is used by WAVE . This is espacially useful, if the UNDUMAG run is time consuming. The file *undumag.wav* contains all voxels of the magnetic setup and their magnetization as calculated by UNDUMAG . WAVE calculates then the contribution of each voxel

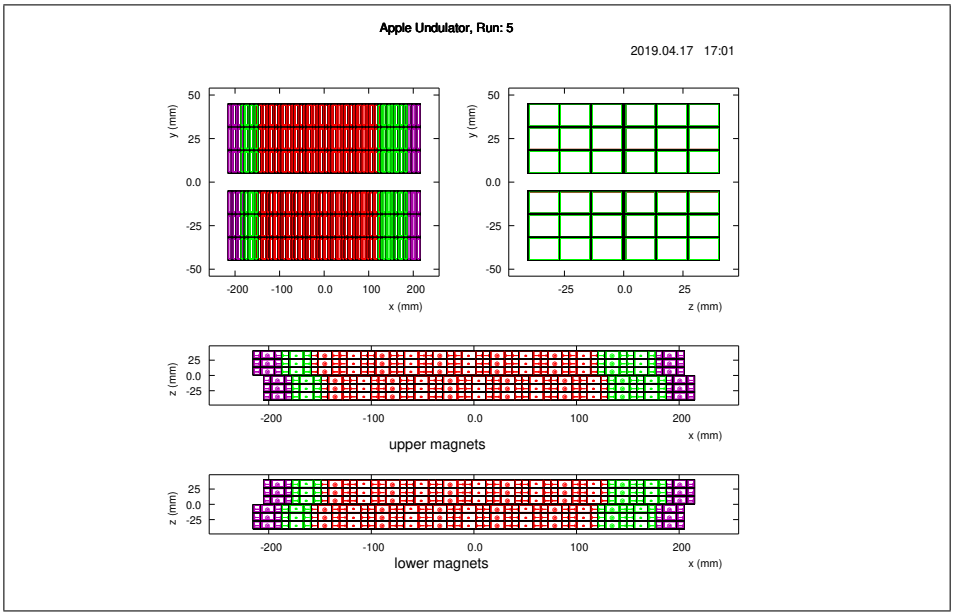


Fig. 36: Example of an APPLE-II undulator

applying the current sheet method. This can also be rather time consuming. Therefore, it might be better to calculate a field map first, with UNDUMAG or WAVE for later use.

In special cases, it might be helpful to edit ***undumag.nam*** and ***undumag.clc*** and not letting WAVE set up them form the templates. This can be achieved by setting ***KBUNDUMAG=-1***. This is equivilent to run UNDUMAG seperately and then setting ***KBUNDUMAG=1***, which also can be achieved by the command:

undumag

The namelist ***UNDUMAGN*** contains the parameters to change the gap, shift of the undulator and other parameters to control UNDUMAG , e.g. the definition of the field map to be calculated. Of course, a field map can also be written by WAVE , check the variable ***IWBMAP*** in ***wave.in***.

The advantage of using UNDUMAG over the REC-undulators of exampel ***4*** is the realistic simulation of the interaction of the magnets. One effect e.g., which can not be simulated with the model of example ***4*** is the effect on the endmagnets and hence on the first and second field integrals when the shift is changed. For more details and options refer to the namelist ***$UNDUMAGN*** in *wave.in*.

Periodic Field expansion

If the segmentation of the items in UNDUMAG is fine, the number of voxels becomes very large and slows down the computation significantly. On the other hand, the periodic field is a genuine characteristic of undulators. Therefor, WAVE offers a method to repeat a part of the field periodically.

Please check the namelist ***$B0SCGLOBN*** in ***wave.in***. This namelist offers options of global modification of the magnetic field configuration, e.g. shifting or scaling the field.

We use this namelist to expand the undulator by twenty periods and set:

KBUNDUMAG=1
XSHIFT=0.56
IPERIODG=-1
XPERWMN=0.0
XPERWMX=1.12
PERIODG=0.056
PEROFFG=0.028

The variable ***IPERIODG=-1*** selects the mode the periodic part of the field is treated.

XPERWMN=0.0 and ***XPERWMX=1.12*** define the region of the periodic part, while ***PERIODG=0.056*** and ***PEROFFG=0.028*** define the period to be repeated.

Since these setting would shift the center of the device, we correct this with ***XSHIFT=0.56*** to keep the center of the undulator at the origin.

We save ***wave.in*** with these settings and run WAVE . The result is the field in Fig. (37).

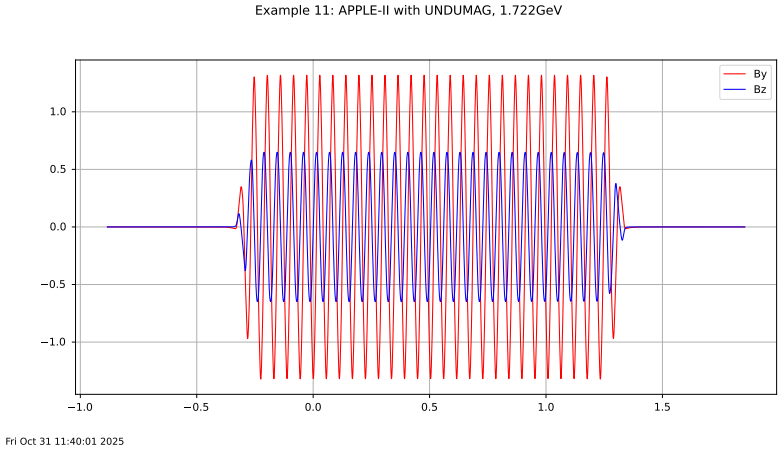


Fig. 37: Field of the expanded undulator

12. Example: Accelerator Physics of Insertion Devices

WAVE has some feature to investigate aspects of the accelerator physics of insertion devices. This chapter presents the most important ones. The results of this calculations really depend on the settings of the parameters. To get the right answers, check the input and the results thoroughly.

12.1 Linear Transfer-Matrix, Multipoles, and Field Maps

Starting point is the Python script *12a*:

```
. $WAVE/shell/set_wave_environment.sh
python3 ../python/wave_example_12a.py
cp $WAVE/input/wave_example_12a.in wave.in
```

The script modifies the file *$WAVE/input/wave.examples* and echos the changes. Please, open *wave.in* and study the corresponding variables and their namelists.

Run with this input WAVE performs three task for an REC-undulator defined in the namelist *RECN*, see example 4.

- Field Map

  The variable *IWBMAP=6* makes WAVE writing a field map for later use. The parameters of the field map are defined in the namelist *BMAPN*. WAVE can read and write field maps of the considered devices. This allows interfacing to other codes, i.e. importing and exporting magnetic field configurations between different programs. In the next subsections we will make use of this field map, mainly to reduce the computing time.

  Note: Different namelists might use different units, especially mm and m or even different coordinate systems, respectively.

  To visualize the field map, start the plotting GUI and enter in the Python shell:

```
nmap = ncread( "nmap", "x:y:z:bx:by:bz", "wave_example12a_bmap.dat", skiphead=6 )
nplot( nmap, "x:by", "abs(x)<0.028 and y==0 and z==0" )
nscan( nmap, "x:by", "abs(x)<0.028 and by>1.0 and y==0 and z==0" )
nplot( nmap, "z*1000:by", "abs(x+0.014)<0.0001" )
setlinecolor('blue')
nplot( nmap, "z*1000:by", "abs(x+0.014)<0.0001 and abs(y-0.0052)<0.0001",  "", "line-
same" )
wave_title()
txyz( "Vert. Field at x = 0 for y = 0/1/2/3/4/5 mm", "z [mm]",  "By [T]" )
```

  The first command creates a Ntuple and reads the field map, skipping the six header lines. Then the field in the center of the undulator is plotted. The scan command is used to locate the maximum of the on-axis field, which is used in the next command. Then the transversal field distribution is plotted (Fig. 38) for this corresponding longitudinal position *x = -0.014 m*, which corresponds to a quarter of the period-length of the undulator.

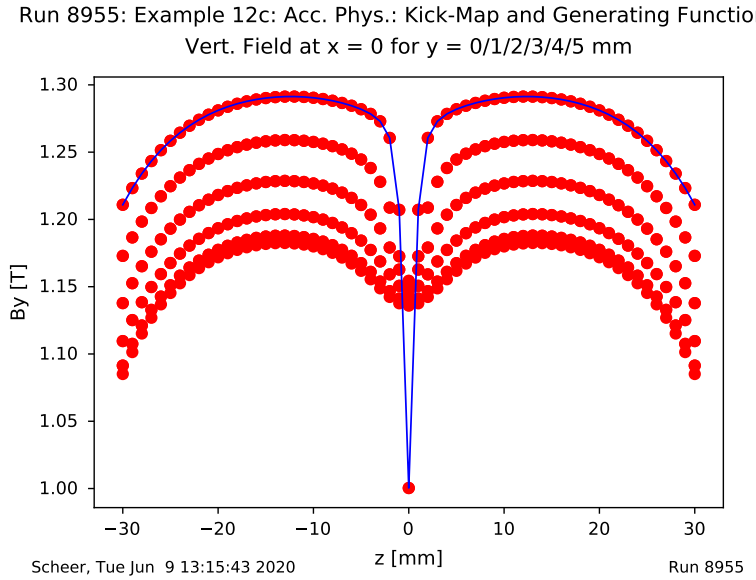


Fig. 38: Transversal distribution of the vertical field in the center of the undulator.

Note the dip in the middle, which is a consequence of the slit between the magnet rows, espacially for ***y = 5.2 mm*** (blue line). This means that the transveral field gradients of the device depend on the transversal position of the beam electrons, thus complicating tracking calculations.

- Quadrupole- and Sextupole Terms

IWBTAB=1 triggers the calculation of the field and the integrals of the field, the quadrupole- and sextupole terms etc. along a straight line. For more details refer to the comments in ***wave.in***. In addition the sextupole terms are written to a ***wbtab_sextupole.dat*** ***IWBTAB=100*** does the same, but with respect to the trajectory rather than a straight line.

By the way: If the field calculation is very time consuming, in many cases the file written due the setting of ***IWBTAB*** is sufficient for later use, e.g. for spectrum caclulations. In such a case, the field can be determined from the file via a very fast spline interpolation, see ***IRBTAB*** in ***wave.in*** . This holds of course also for other tabulated fields ***B_y(x)***, e.g. from measuements. It is also possible to provide this way two field components ***B_y(x)*** and ***Bz(x)***, see ***IRBTABZY*** in ***wave.in*** .

- Linear Transfer Matrix

DELTAZ=0.0001 and the other variables of the namelist ***TRALINN*** trigger the simple calculation of the linear transfer-matrix by tracking four principal trajectories through the device. The results are given in ***wave.out***. The matrix is also written to ***tralin.wav***. We copy this file:

cp tralin.wav tralin_rec.wav

for the next subsection.

The second and third item of this run are not only excecuted for their results, but also to compare with the later runs, when the field map will be used. This way, one might get an impression of the accuracy and uncertaincies related to the use of a field map instead of the "true" field.

12.2 The Linear Transfer-Matrix from the Field Map

We excute the Python script 12b:

```
. $WAVE/shell/set_wave_environment.sh
python3 ../python/wave_example_12b.py
cp $WAVE/input/wave_example_12b.in wave.in
```

and run WAVE . We compare ***tralin.wav*** with the file of the previous subsection ***tralin_rec.wav*** . Have a look on the matrix terms

```
m21 = -0.01166
m21 = -0.00865
```

and

```
m43 = -0.02564
m43 = -0.02500
```

,

which represent the focal strength, i.e. the inverse of the focal-lenght of the horizontal and vertical focussing. If the required accuracy is not reached, the grid of the field map must be chosen finer, or the "true" field model must be used.

12.3 The Kick-Map and Generating Functions

Generating functions (GF) are a poweful tool for tracking calculations, since they allow symplectic tracking with very few steps through a whole insertion device. The GF take into account higher order effects, i.e. they represent the device much better than the linear transfer-matrix. To calculate a Taylor expansion of the GF F_2 numerically, we excute the Python script 12c:

```
. $WAVE/shell/set_wave_environment.sh
python3 ../python/wave_example_12c.py
cp $WAVE/input/wave_example_12c.in wave.in
```

The GF is now calculated in two steps when WAVE is run. In the first step the variable ***IOPTIC = 1*** triggers the subroutine ***OPTI***, wchich tracks a bundle of electrons as defined in the namelist ***$OPTIK*** through the devices. This step can be very time consuming.

In the second step, the GF is calculated. This is triggered by the variable ***IGENFUN = 1*** (see also namelist ***$TRANPON***).

Open ***wave.out*** and study the output of the subroutine ***TRANPOLY***, which fits the coefficients of the GF. Please compare the differences between the linear transfer matrices, quadrupole matrix derived from the GF and matrix calculated in the subsections above. For real tracking calculations using the GF, one must check, if the field map is good enough or the "true" field should be used. And one also

should vary the bunch of tracks, the phase-space etc. and study the effects on the GF carefully... This is beyond the scope of this example.

We have a look on the tracks used for the GF:

```
wplot
nlist()
ninfo("n501")
ndump("n501","zi*1000:yi*1000:zpf*1000:ypf*1000","","wave_example_12c_kick-
map.dat")
```

The command ***nlist()*** gives an overview of the Ntuples read from ***WAVE.mhb***. We note the Ntuple ***n501*** and print the summary using the command ***ninfo*** . Now, we can plot the variables or write them to a kick-map file using the ***ndump*** function. The full syntax of this procedure can be seen by entering:

```
ndump()
```

If no file name is given, the default ***ndump.dat*** is chosen.

This example also demonstrates a convient way, to extract and convert data from ***WAVE.mhb*** for later use. However, since the Ntuples and histograms are nothing else then ***Pandas Dataframes*** , you can directly use the commands provide by the Python package ***Pandas*** and ***Matplotlib*** .

13. Example: User Interfaces

Although WAVE offers a variety of options, sometimes it's necessary to use some special input data, to write a user-defined output, to perform a complicated fit, or to provide some unusual field map, or ...

There are some routines to be provided by the user:

- ***subroutine uname***

  This routine allows to provide user input stuff or access and modify the variables of WAVE.

- ***subroutine uout***

  You can use this routine to access the variables of WAVE at the end of the run and to use it for your own output.

- ***subroutine ustep(x,y,z,vx,vy,vz,vxp,vyp,vzp,dt,gamma,dgamma)***

  This routine is called during each tracking step. You can manipulate the given arguments.

- ***subroutine bextern(x,y,z,bx,by,bz,ax,ay,az)***

  If you want to provide your own magnetic field, set *(*KBEXTERN=1). The routine must return the magnetic field and vector potential (if needed) for the point ***(x,y,z)*** .

- ***subroutine ubmask(x,y,z,bxmask,bymask,bzmask)***

  This routine multiplies the magnetic field (***IBMASK=-100***) or adds the masking field (***IBMASK=-200***). Please check the namelist ***$B0SCBLOBN*** in ***wave.in***.

14. Example: Effective Undulator Source

Since the synchrotron radiation of an undulator is coherently emitted from the whole trajectory through the device, an effective source can be defined by propagating back the field distributions calculated in a pinhole down-stream of the device. This can be done with WAVE by setting

IPHASE=1

To get in addition the Wigner-distributions of the source set e.g.

IWIGNER=-1

The parameters for the source plane and the Wigner-distributions are defined in the namelist ***$PHASEN***. Please, have a look.

We enter in the shell:

```
. $WAVE/shell/set_wave_environment.sh
python3 ../python/wave_example_14.py
cp $WAVE/input/wave_example_14.in wave.in
```

WAVE writes in this example a file ***wigner.wav***, which contains the Wigner-distribution.

Of course, the set-up can and plotting can be done within the GUI ***WAVESHOP***. The radiaton cone in the pinhole down-stream of the undulator is shown in Fig. 39

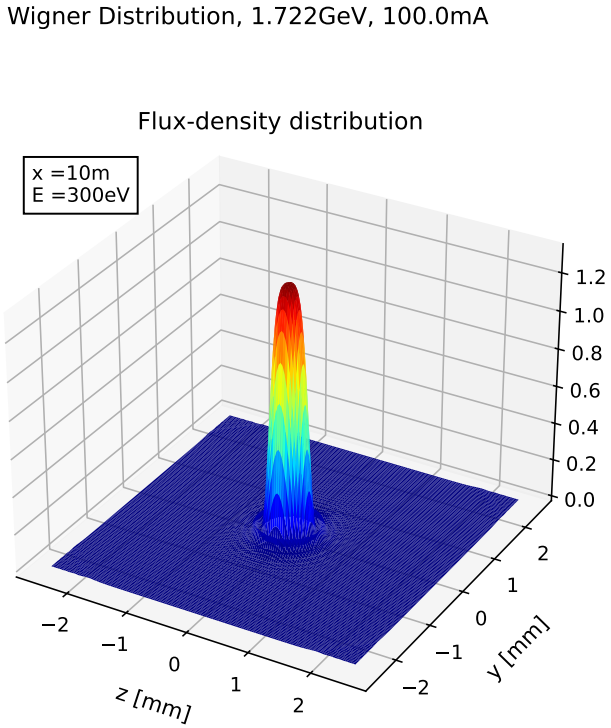


Fig. 39: Radiation cone of the undulator

The fields of this cone a propagated back to the center of the undulator. Fig. 40 shows the real part of the horizontal field.

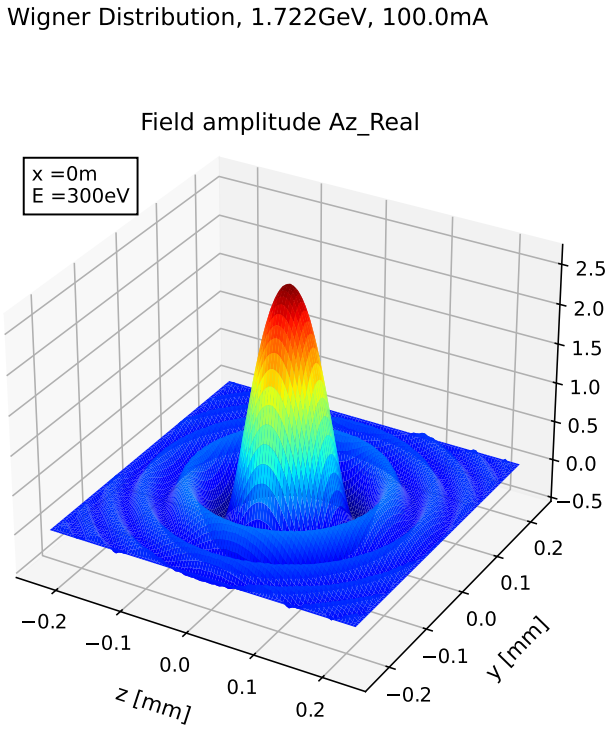


Fig. 40: Radiation cone of the undulator

From the back-propagated field, the Wigner-distrubtion are calculated. Fig. 41 shows the distribution W_{zz} of the horizontal field in the $z - \Theta_z$ - plane.

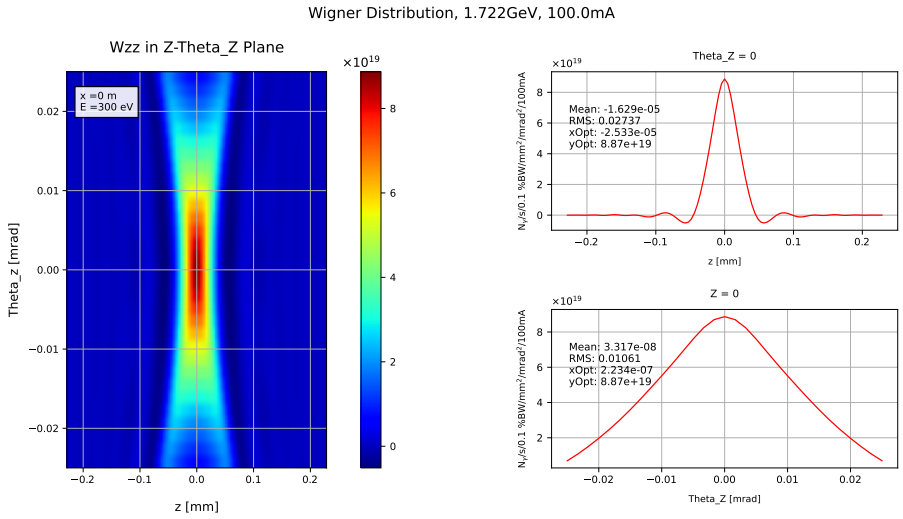


Fig. 41: Wigner-distribution W_{zz} in the $z - \Theta_z$ - plane.

Often, ones want to plot only a 2d-cut of the 4d distribution. Then it is useful, to limit the phasespace to the considered range, since the calculations can be very time-consuming. Therefore

NWIGTHETAY=1

has been chosen for this example.

To include the energy-spread of the electron beam, one must set ***ESPREAD*** and ***NWIGEFOLD***, e.g.

NWIGEFOLD=15

This triggers 15 WAVE - runs with beam energies distributed according to ***ESPREAD*** and calculates the weighted integral of the contributions and writes the results to a file named ***wigner.wef*** . This can be rather time consuming. The beam emittance can be taken into account by folding the results accordingly. This is not yet implemented in WAVE , but can be performed by the user on base of the data in ***wigner.wef***. This file is read by ***WAVESHOP*** , so you can see the variables of the file in the n-tuple ***nwef***.

References

[1] M. Scheer, "Beschleunigerphysik und radiometrische Eigenschaften supraleitender Wellenlängenschieber", Dissertation, Humboldt-Universität zu Berlin, Mathematisch-Naturwissenschaftliche Fakultät I , 2008, urn:nbn:de:kobv:11-10096299

M. Scheer, WAVE - A Computer Code for the Tracking of Electrons through Magnetic Fields and the Calculation of Spontaneous Synchrotron Radiation, http://accelconf.web.cern.ch/AccelConf/ICAP2012/papers/tuacc2.pdf

[2] Michael Scheer, "UNDUMAG - a new computer code to calculate the magnetic properties of undulators", Proceedings of IPAC, Copenhagen, Denmark, 2017, pp 3071-3073